

4. **Support semantic consistency** across languages (embedding-based similarity per locale).
5. **Enable full multilingual publishing workflows** directly in the CMS.

This is the layer that makes Rosotravel a **multi-language global OTA**, not just an English-only site.

I.02 Translation Engine Specification

Translations use **OpenAI GPT-4o/4.1** with semantic-awareness.

Translation rules (mandatory):

- Never translate:
 - factual fields
 - ecommerce properties (price, currency, meeting point)
 - place names (detected via Named-Entity Recognition)
 - keywords marked as “locked”
- Always translate + adapt:
 - titles
 - descriptions
 - highlights
 - itinerary steps
 - FAQs
 - meta title & meta description
 - schema text fields (except factual properties)

Token preservation rules:

- All SEO tokens ({{City_Name}}, {{Attraction_Name}}) remain EXACTLY as tokens during translation.
 - The model sees:
“Translate but keep {{City_Name}} unchanged.”
-

I.03 Language Expansion Strategy (Mandatory)

Supported languages (initial):

EN, DE, FR, IT, ES, PL, NL

Future-ready architecture supports:

PT, SV, NO, CS, RO, HU, JP, AR, ZH, KR

Language Release Strategy

- English = canonical source.
 - Other languages follow a **dependent publishing pipeline**:
EN → draft_DE, draft_FR, draft_IT ...
 - Languages only publish when EN is validated.
-

I.04 Keyword Ownership Across Languages

This is **critical** for SEO stability.

Rule 1 — Keyword Locking Inherits Downstream Languages

If EN keyword is locked:

Example:

schonbrunn palace guided tour → locked to Product A
→ German / Italian keywords “schönbrunn führung” / “tour guidato palazzo schönbrunn”
must also map to Product A.

Rule 2 — No language may introduce new primary keywords

All foreign-language primary keywords are derived from:

- direct translation of EN locked keyword

- validated via embeddings to prevent drift

Rule 3 — Local-language volume checks (optional, not required)

If SEMrush DE/IT/FR is added later → system checks for local equivalents.

I.05 Multi-Language URL Strategy

You asked earlier whether to use different ccTLDs — the final spec:

Canonical URL always uses EN

Example:

<https://www.rosotravel.com/italy/rome/colosseum-guided-tour>

Localized URL pattern (subfolder-based):

[/de/italien/rom/kolosseum-geführte-tour](#)

[/fr/italie/rome/visite-guidée-colisée](#)

[/it/italia/roma/tour-guidato-colosseo](#)

Rules:

- Localized slugs are allowed and recommended.
 - Canonical still points to EN.
 - Localized pages get hreflang pairs.
-

I.06 Hreflang Generation Rules

For every published URL, system must generate:

G.04 Canonical, hreflang & Meta Robots

G.04.1 Canonical rules

- Every indexable page must have exactly one canonical URL.

- Canonical must always be the clean URL without query parameters, filters or tracking.
- For dynamic filter pages (`?duration=2h&language=en`), canonical always points to the base page without parameters.
- When migrating legacy URLs to new structure, old URLs must 301 redirect to new canonical URLs.

G.04.2 hreflang rules

- For each page that exists in multiple languages, hreflang tags must be generated:
 - `x-default` for global default (US-English).
 - One `<link rel="alternate" hreflang="xx-YY" href="...">` per locale.
- Full list of hreflangs to implement -
- Mapping between locales and URLs must come from CMS language/locale table, not guessed.
- hreflang must never mix different page types (product en ↔ product de, not product ↔ attraction).

```
<link rel="alternate" hreflang="en"
href="https://rosotravel.com/..." />
<link rel="alternate" hreflang="de"
href="https://rosotravel.com/de/..." />
<link rel="alternate" hreflang="fr"
href="https://rosotravel.com/fr/..." />
<link rel="alternate" hreflang="x-default"
href="https://rosotravel.com/..." />
```

CMS must show a “Localizations panel”:

- Language
- URL slug
- Status (draft, pending QA, published)
- Translation completeness

- Preview button
 - Regenerate translation
-

I.07 Translation Pipeline Step-by-Step

Below is the **exact position** of the translation step inside the global pipeline:

1. EN final version published

All sections locked; schema inserted; embeddings saved.

2. Translation job triggered automatically

Queued per batch.

3. Pre-translation preparation

- strip factual blocks
- extract NER entities (place names)
- tag locked keywords
- isolate schema text fields

4. GPT translation for each field

Translation prompt ensures:

- semantic adaptation
- UX rewriting
- SEO intent preservation
- no keyword collisions

5. Schema translation

Only text nodes translate; factual nodes preserved.

Example preserved:

```
"geo": { "lat": 41.8902, "lng": 12.4922 }
```

Example translated:

```
"description": "Besuchen Sie das Kolosseum mit einem lokalen  
Expertenführer..."
```

6. Localization QA rules (automated)

AI validators check:

- token correctness
- keyword presence
- keyword similarity
- length limits
- correctness of slugs
- no hallucinated facts

7. Human sampling (optional)

3–5% for major markets (DE, FR).

8. Publish localized versions

- Write to CMS
- Insert hreflang
- Update sitemaps
- Re-embed content for each locale

I.08 Embeddings Per Language

Every localized version must generate a **separate embedding**:

Collections in Qdrant:

pages_vectors_en
pages_vectors_de
pages_vectors_fr
...

Why?

- prevent cross-language collisions
 - support multi-language near-page detection
 - enable language-specific duplication prevention
 - improve LLM understanding
-

I.09 Multi-Language Schema Rules (Detailed)

Fields translated

- `description`
- `name`
- `review.body`
- `faq.question`
- `faq.answer`
- `touristTrip.itinerary`

Fields NEVER translated

- `geo`
- `address`
- `openingHoursSpecification`

- `price`
- `currency`
- `tourBookingPage`
- `sameAs`
- `identifier`

Fields that adapt to locale

- `@language`
- `inLanguage`

I.10 Multi-Language Internal Linking Rules

Internal links must:

1. **Link to the same language version**
 DE → DE, FR → FR
 Never: DE → EN
2. **Cross-language logic only for canonical linking**
3. Generated dynamically via CMS using:
`{{localized_url(entity, lang)}}`

I.11 CMS Requirements for Localization Layer

The CMS must contain a **Localization tab** for every entity:

Localization Panel

Field	Description
Language	EN, DE, FR, IT...

Status	Draft / Needs QA / Published
Slug	Editable localized slug
Preview	Preview localized page
Regenerate	Regenerate translation
Locked Keywords	Display inherited SEO tokens
Factual Fields	Gray-out (non-editable)
Manual Override	Allow editor to rewrite localized version

Localized WYSIWYG

- Shows EN side-by-side with target language
- Non-editable factual blocks
- Highlight translated vs auto-translated sections

I.12 Error Handling & Fallback

Problem: Model hallucinated location names

→ Auto-rejection → regenerate using Azure Translator → flag for review

Problem: Keyword mismatch across languages

→ Block publish → requires human approval

Problem: Hreflang inconsistency

→ Rebuild hreflang table automatically → retry publish

Problem: Translation API failure

→ Queue retry

→ Fallback to Azure

I.13 Real Example (Live-like)

EN Input

Title: Skip-the-Line Colosseum Guided Tour

Short Description: Enjoy priority access to the Colosseum with a licensed guide.

FAQ:

Q: How long is the tour?

A: The tour lasts approximately 2 hours.

DE Output

Titel: Geführte Kolosseum-Tour ohne Anstehen

Kurzbeschreibung: Genießen Sie bevorzugten Zugang zum Kolosseum mit einem lizenzierten Guide.

FAQ:

F: Wie lange dauert die Tour?

A: Die Tour dauert etwa 2 Stunden.

Preserved factual fields

- price
- meeting point
- lat/lng
- address

Translated schema fields

"@type": "TouristTrip",

"name": "Geführte Kolosseum-Tour ohne Anstehen",

"description": "Genießen Sie bevorzugten Zugang ..."

"inLanguage": "de"

I.14 How This Fits Into the Full Pipeline

Integrated version:

RAW ingestion →

Differentiation →

SEO keyword allocation →

AI drafting (EN) →
Schema drafting →
AI validation →
FAQ retrieval + FAQ drafting →
Human QA →
Publishing (EN) →
Final embeddings →
Translation engine →
Localization QA →
Publishing localized versions →
Localized embeddings →
Hreflang + sitemap updates →
Monitoring →
Dynamic near-pages (EN only) →
Repeat

4.19 Post-Publish Optimization Loop (Monitoring → Signals → Job Queue → Daily Publish Limits → 5-Week Near-Pages)

Tu ma być: monitoring → inbox signals → jobs → throttling/daily limits → near-pages window + gates.

Uwaga spójności: publishing queue/caps są tu SSOT, a 4.17 tylko referencja do limitów.

L. Monitoring, Dashboards & Operational Tooling

01. AUTOMATIC OPTIMIZATION ENGINE (BACKEND, NO HUMAN INVOLVEMENT)

Automatic Trigger System (Runs Daily, Uses GSC + GA4 + Similarity + Schema Validity)

AI Optimization Trigger Rules (GSC Data)

CTR-based

Condition	Trigger
CTR < 1.2% with ≥ 200 impressions in last 30 days	Add to "Optimization Queue" → Regenerate Meta Title + Meta Description (AI)

CTR between 1.2% and 2.0% Suggest manual review; non-critical alert

Position-based

Condition	Trigger
Average Position drops by >3 over 14 days	Regenerate Intro Paragraph (AI) + highlight revision
Average Position stagnates for 30 days	Suggest additional internal links; no automatic regeneration

Impressions-based

Condition	Trigger
0 impressions for 30 days	Mark page as “Dead Page” and add to queue for re-optimization or merge
Impressions < 20 for 30 days	Suggest near-page or keyword expansion candidate

Similarity-based

Condition	Trigger
Page similarity ≥ 0.90	Canonical review alert (CMS notification)
$0.75 \leq \text{similarity} < 0.90$	“Merge keyword clusters” → manual decision

Schema / FAQ triggers

Condition	Trigger
Missing schema or schema validation error	Schema regeneration (AI auto-run)
FAQ similarity > 0.85 to another FAQ	FAQ duplication alert + manual review required

1.2 UX Optimization Triggers (GA4 Data)

(Exact thresholds preserved)

Bounce Rate

Condition	Trigger
Bounce Rate > 55%	AI generates alternative intro + alternative highlights

Dwell Time

Condition	Trigger
Avg engagement time < 15 seconds	AI regenerates stronger intro, improves first 150 words

Scroll Depth

Condition	Trigger
Scroll depth < 35%	System inserts additional images, dividers, short paragraphs, bullet summaries

Conversion Rate

Condition	Trigger
Conversion drops > 20% (MoM)	Add to A/B Test Candidate queue (meta + intro variants)

2. WEEKLY CLASSIFICATION ENGINE (LANGCHAIN)

Runs every **7 days** on every **indexable page**.

Classification Types (FULL LOGIC)

Category	Meaning / System Action
OK / Stable	No action
Needs Optimization	Auto-run AI regeneration based on thresholds
A/B Candidate	AI generates two variants (meta or intro)
Human Review	Edge case → manual approval required
Dead Page	No impressions / poor metrics → merge/noindex candidate
Trending Topic Candidate	Eligible for automatic near-page creation

Stored in:

`weekly_page_classification`

Feeds:

- Optimization Inbox
 - Near-page queue
 - SEO Dashboard
-

3. AUTOMATIC ACTIONS (BACKEND PIPELINE EXECUTION)

3.1 Automatic Regeneration Jobs

Triggered automatically when thresholds fail:

- Regenerate Meta Title + Meta Description
- Regenerate Intro Paragraph
- Regenerate Highlights
- Generate New FAQ Entries
- Regenerate Schema
- Generate two variants for A/B tests
- Insert images / dividers / bullet-summaries
- Trigger canonical review
- Mark page as “dead page”

3.1.1 Dynamic Experience Engine Jobs (MVP, non-indexable UI layer)

These jobs generate and maintain *segment variants* that affect only on-page UI (not meta, not H1/H2, not canonical body SEO text). Variants are stored per page in CMS fields and selected at runtime based on the user’s inferred segment.

A) Dynamic Snippet Variants (Primary, mandatory for every indexable page type)

Trigger: first publish + any “regen snippet” action + when base snippet changes.

Output fields (stored per entity/page in CMS):

- snippet_first_time_visitor, (it is default audience)
- family_traveler,
- couple_traveler,
- comfort_easy_pace_traveler,
- solo_social_traveler,
- interest_deep_dive_traveler,

- active_adventure_traveler

Rules: each field is 180–260 characters; one paragraph; neutral, high-density; no hype; not used in meta.

Runtime: if no segment detected → render snippet_default.

B) Review Summary Variants (Primary, product/city/attraction/region/country)

Trigger: new reviews ingested OR weekly refresh OR manual “rebuild review summaries”.

Output fields (stored per entity/page in CMS):

- review_summary_first_time_visitor, (it is default audience)
- family_traveler,
- couple_traveler,
- comfort_easy_pace_traveler,
- solo_social_traveler,
- interest_deep_dive_traveler,
- Active_adventure_traveler

Rules: each summary is a short, non-quoted theme summary derived from review clusters (no fabricated facts).

Runtime: rendered under rating summary; selected by segment; if unavailable → fallback to default.

C) Ordering Profiles (Listings/Recs, must match snippet segment set)

Trigger: weekly recalculation + when inventory changes materially (availability/price/rating deltas) + manual “rebuild ordering profiles”.

Stored config (system + CMS override):

- ordering_profile_first_time_visitor, (it is default audience)
- family_traveler,
- couple_traveler,
- comfort_easy_pace_traveler,
- solo_social_traveler,
- interest_deep_dive_traveler,
- active_adventure_traveler

Scope: controls ordering only (not indexable URLs):

- city “Top tours” feed ordering
- things-to-do “Top activities” feed ordering
- related tours carousel ordering

- bundles card ordering on city/attraction pages (if multiple bundles exist for a segment)
Fallback: default ordering_profile_default.

3.1.2 Human-Face / Live Experience Layer Jobs (MVP)

These jobs keep “freshness + real people” blocks updated and consistent with review/tour-date logic.

A) Last Tour Snapshot Builder (product pages)

Trigger: new review ingested OR new tour run event OR daily refresh.

Inputs: reviews feed (must include tour_date), guide assignment (if Rosotravel-owned), uploaded guest photos with tour_date tags (if available).

Output (system fields used by FE):

- last_tour_date_time
- last_tour_guide_display (name + credentials if available)
- last_tour_group_size (if available)
- last_tour_segment_mix (if derivable)
- last_tour_mood_score (post-tour rating aggregate for that tour_date window)
- last_tour_guests_loved_topics[] (2–3 themes, no quotes)
- last_tour_photos[] (filtered gallery references by tour_date)
Fallback rules:
 - if no recent tour_date exists → show “Last tour: {month/year}” + optional “Last 30 days summary” (configurable).

B) “This Week in {City}” Live Block Builder (city pages) / “Recent at {Attraction}” (attraction pages)

Trigger: weekly refresh + new reviews ingested.

Output: tours_run_count, avg_rating_week, top_guides_week[], guests_loved_week_topics[], recent_guest_photos_strip[] (optional).

3.1.3 Intent & Variant Telemetry Jobs (required for MVP tuning)

Trigger: continuous event ingestion + daily rollups.

Store (system tables):

- session_intent_signals (raw events; short retention)
- session_segment_inference (final segment + confidence + reasons)

- variant_impressions (which snippet/review_summary/ordering_profile was shown)
- variant_performance_rollups (CTR to booking panel, scroll depth, add-to-wishlist, email captures)

This telemetry must be used only to improve the on-page semantic layer and ordering (not to alter keyword clustering or canonical SEO ownership).

3.2 Backpressure Logic

If job queues exceed limits:

- regeneration slows down
- low-priority regenerations postponed
- DevOps alert sent (rate limits, Qdrant latency, API quota, etc.)

7. NEAR-PAGE GENERATION

7.1 5-Week Trigger Window

Every published page enters a 5-week “learning window”.

After 5 weeks → system checks:

- GSC new queries
- semantic gap
- impressions rising / falling
- keyword similarity
- unused keyword opportunities

G.07.12 Near-Page Cannibalization Gate (Mandatory, Pre-Queue)

Before a keyword/topic can enter the near-page queue, the system **MUST** run a deterministic cannibalization check against **all stronger page types**:

The candidate MUST be rejected (no near-page) if ANY is true:

1. **Transactional lock conflict**

candidate keyword similarity $\geq$ `sim_variant_low` to any locked product keyword cluster.

2. **Covered by existing owner**

candidate similarity $\geq$ `sim_borderline_low` to:

- city page owned cluster, OR
- category/things-to-do owned cluster, OR
- attraction owned cluster

3. **Resolvable as FAQ/section instead of URL**

candidate intent is informational/commercial AND the page can absorb it as:

- FAQ question (preferred) OR
dedicated “mini-section” (preferred)
without changing primary keyword / intent / H1.

Only if ALL pass → candidate may be queued as `near_page_draft`.

Hard hierarchy rule (must be encoded): Resolve by absorption order before URL creation:

1. Product (if transactional) → never near-page
2. City / Category
3. Attraction
4. FAQ / Section injection
5. Near-page (last resort)

7.2 Near-Page Similarity Decision Tree (Thresholds in D.05)

Similarity band is determined by D.05 Keyword Similarity Thresholds (single source of truth).

Similarity band
(from D.05)

Action (near-page policy)

Duplicate
phrasing

Reject. Do not create a near-page. No content change triggered.

Variant phrasing	Reject for new URL. Attach only to the existing owner as a secondary keyword.
Borderline topic	Create <code>topic_candidate</code> → requires SEO Admin decision: prefer FAQ/section absorption on the owner. Near-page draft allowed only if cannibalization + volume gates pass.
New topic	Eligible to create a near-page draft (still must pass intent allowlist + cannibalization + volume gates).

Hard rule: A near-page can never be created from Variant phrasing.

Near-Page Eligibility Gates (Mandatory)

A near-page draft is allowed only if ALL conditions are met:

1. **Intent allowlist:** `semrush_intent` $\in$ {`Informational`, `Commercial`}
2. **Similarity band:** band is **Borderline** or **New topic** (per D.05); Variant is forbidden for URL creation
3. **Cannibalization gate:** candidate must not collide with:
 - locked transactional product clusters
 - existing city/category/attraction owners (absorb as FAQ/section first)
4. **Demand gate (dynamic):** candidate must satisfy either:
 - **Standard gate:** `semrush_volume` $\geq$ `near_page_min_volume_standard` (default **20**), OR
 - **Long-tail gate (GSC-proven):** `semrush_volume` $\geq$ `near_page_min_volume_longtail` (default **10**) AND `gsc_impressions_28d` $\geq$ `near_page_min_gsc_impressions_28d` (default **25**)
5. **No existing ownership:** keyword/topic is not already owned in `keyword_map`

CONFIG

- `near_page_min_volume_standard = 20`
 - `near_page_min_volume_longtail = 10`
 - `near_page_min_gsc_impressions_28d = 25`
-

Single-Record Processing After Batch Activation (Late Products, No Ownership Reclaim)

7.22 Incremental Supply After Initial Batch (New Products + New Attractions)

This section defines deterministic behavior for products and attractions added after the initial city batch has been published.

A) New Product Added to an ACTIVE City

If `city.status == ACTIVE`:

- product may be drafted and published immediately
- keyword assignment **MUST** use `keyword_map` ownership rules (no free-form discovery)
- SEMrush calls are **optional** and limited to verifying the chosen product primary keyword via `batch_keyword_overview`

CONFIG

- `new_product_semrush_check_enabled = true`
- `new_product_semrush_check_mode = "batch_keyword_overview_only"`

B) New Product References an Attraction That Does Not Yet Exist

If product includes an attraction token/entity not present in registry:

- create `attraction.status = DRAFT`
- attach the product to the attraction entity for UX grouping
- attraction SEO page **MUST NOT** auto-publish as indexable

- attraction page starts as:
 - `index_state = noindex, follow`
 - `canonical_target = parent city/attraction owner (per governance)`

Marketer action required

Marketer must explicitly set attraction to:

- `ACTIVE` (eligible for publishing)
- `SEO_ENABLED` (eligible for indexing gates)

C) Micro-Discovery for a Newly Activated Attraction (Cost-Safe)

When a marketer activates a new attraction:

- generate a minimal seed set:
 - `{attraction_name}`
 - `{attraction_name} tickets`
 - `{attraction_name} guided tour`
- run `phrase_fullsearch` with `display_limit = 100`
- do NOT run Related/Questions unless the attraction is in a top-demand allowlist

CONFIG

- `new_attraction_seed_count = 3`
- `new_attraction_fullsearch_display_limit = 100`
- `new_attraction_related_enabled = false` (default)
- `new_attraction_questions_enabled = false` (default)

D) Incremental City “Micro-Batch” (Optional, Operational Simplicity)

To avoid re-running the full batch for a city, the system may run a micro-batch only if:

- at least `micro_batch_min_new_products` new products were added since last run OR
- `micro_batch_max_wait_days` reached

Outputs of micro-batch are limited to:

- keyword attachment
- FAQ/section expansion
- near-page candidate queueing (still governed by 5-week + DQCP)

CONFIG

- `micro_batch_min_new_products = 40`
- `micro_batch_max_wait_days = 30`

Hard rule: micro-batch MUST NOT create category explosions or mass near-pages.

DEE rules optimization:

If a page underperforms:

- CTR triggers meta rewrite
- bounce triggers intro rewrite
- low scroll triggers highlight rewrite
- stagnation triggers additional blocks
- cannibalization triggers keyword merge
- new queries trigger near-page creation

This makes the platform self-improving.

4.20 LLM & MCP Outputs Contract (AI Sitemap, LLM Feed, /ai-query, MCP Read-Only Safety)

Tu ma być: AI sitemap + LLM visibility feed contract + /ai-query + MCP tools + safety + red lines (co nigdy nie wychodzi).

01. Machine-to-Machine Layer (Search Engines + Agents + LLMs)

Rosotravel is designed to be equally readable by humans and machines. Every published page must expose:

- a stable entity identity (Main_Entity_ID),
- deterministic structured data (schema + relations),
- canonical URLs and language signals,
- and a read-only machine interface (AI feed + MCP tools) that mirrors the same entities and relations as the website.

This ensures bots can reliably understand destinations, compare offers, and complete bookings without ambiguity or unsafe actions.

Core MCP Tools (MVP Scope)

- MCP tools must be built on top of the same governed datasets (inventory feed + publishing filters).

1) search_tours

Purpose

Return a structured list of tours matching a location and simple filters.

Parameters

- {
- "country": "Austria",
- "city": "Vienna",
- "categories": ["tickets", "walking"],
- "max_results": 10
- }

Response

- {
- "results": [- {
- "entity_id": "rt_9876",
- "name": "Schönbrunn Palace Skip-the-Line Tour",

- `"canonical_url":
"https://rosotravel.com/en/austria/vienna/schonbrunn-palace-to
ur/",`
- `"price_from": 39,`
- `"currency": "EUR",`
- `"duration_minutes": 150,`
- `"rating_average": 4.8,`
- `"source_type": "imported"`
- `}`
- `]`
- `}`

Rules:

- Results include only published, indexable products
- Ordering is deterministic (rating + popularity)
- No DEE logic

2) get_tour

Purpose

Fetch full structured details for a single tour.

Parameters

- `{`
- `"entity_id": "rt_9876"`
- `}`

Response

- Mirrors product schema from `/api/tours/feed`
- Includes:
 - name

- short_summary
 - highlights
 - key_faq
 - trust signals (rating if allowed)
 - canonical_url
 - Excludes:
 - availability calendars
 - variant matrices
 - supplier metadata
-

3) get_availability_link

Purpose

Return a safe booking handoff URL.

Parameters

- {
- "entity_id": "rt_9876",
- "date": "2026-06-12"
- }

Response

- {
- "booking_url":
 "https://rosotravel.com/en/austria/vienna/schonbrunn-palace-to
 ur/?date=2026-06-12"
- }

Rules:

- Always Rosotravel domain
 - Never supplier endpoints
 - Date parameter is optional hint, not validation
-

4) get_bundles

Purpose

Expose curated city itineraries to LLM planners.

Parameters

- {
- "city": "Vienna",
- "days": 2
- }

Response

- {
- "bundles": [- {
- "name": "Essential Vienna in 2 Days",
- "canonical_url":
 "https://rosotravel.com/en/austria/vienna/itineraries/2-days/"
- ,
- "highlights": [- "Schönbrunn Palace",
- "Historic City Walk",
- "Danube River Cruise"
-]
- }
-]
- }

Rules:

- Bundles are treated as first-class entities
 - No dynamic segmentation
 - No personalization
 - Same publishing gate as products
-

Data Governance Rules

The MCP Server must:

- enforce the same publishing filters as `/api/tours/feed`
- exclude:
 - drafts
 - noindex pages
 - experimental URLs
- respect keyword ownership and canonical mapping

If an entity is not visible in feeds → it is not visible in MCP.

Rate Limiting & Safety

- Global rate limits per IP / API key
- No authentication required for MVP
- Logging:
 - tool name
 - parameters
 - response size

- No PII logged or returned
-

Relationship to ChatGPT Apps SDK

This MCP Server:

- is compatible with ChatGPT Apps SDK
- does **not** require ChatGPT UI embedding
- can be reused by:
 - Perplexity
 - internal planners
 - future partners

Important:

MCP ≠ UI.

UI embedding is optional and out of MVP scope.

Phasing Recommendation

Phase 1 (MVP)

- Deploy MCP Server
- Tools: search_tours, get_tour, get_availability_link
- No UI integration

Phase 2

- Add get_bundles
- Optional Apps SDK connection

Phase 3

- Agent-assisted planning (handoff only, still no booking)
-

Strategic Outcome

- Rosotravel becomes **agent-readable**
- LLMs can plan trips using Rosotravel as a trusted source
- Zero SEO risk
- Zero supplier API leakage
- Clear differentiation vs classic OTAs

G.07.2 CMS configuration for filters

- There must be an admin config screen defining which parameters are:
 - SEO: can produce unique indexable pages (rare).
 - UX only: always canonical to parent.
- This config must feed into Publishing Layer to generate correct canonical + meta robots tags.

3.8 Automatic Files for AI

Purpose

Provide machine-friendly entry points for LLMs and search.

Data/Schema

- **ai-sitemap.xml / llm_index.xml**: fields **topic**, **entity**, **region**, **summary** (refresh every 24h).
- **ai-summary.json**: brand, regions, coverage, offerings.

- **robots.txt**: allow access to AI files.

Backend

- Generate and refresh files; track **lastmod**; validate URLs.

Front-End

- Host files at root; ensure discoverability (robots, footer links optional).

APIs/Integration

- Endpoint for summary **GET /ai-summary.json**; sitemaps at well-known paths.

Validation

- All links resolve 200; size under practical limits; no PII.

KPIs

- Crawl frequency; external citations by LLMs.

Acceptance (DoD)

- Files present, current, and valid.

G.06.2 AI sitemap & AI summary feeds Publishing Layer must also maintain:

- **/ai-sitemap/** – machine-readable sitemap for LLMs (JSON or XML) containing:

- URL
- entity_type
- primary_keyword
- geo (if applicable)
- isPartOf / hasPart / nearby references
- /ai-summary.json – compressed high-level summaries per entity:
 - short summary text
 - 3–5 highlights
 - key FAQs
 - structured fields for quick ingestion by ChatGPT, Perplexity, etc.

G.06.2.1 LLM Visibility API Feed Contract

Purpose

The goal of the LLM Visibility API Feed is to make Rosotravel **easily readable, indexable, and usable by all LLM systems and agentic frameworks** (ChatGPT, Gemini, Perplexity, Copilot, future autonomous agents), without scraping HTML and without exposing proprietary or partner APIs.

This layer acts as:

- a **machine-readable knowledge surface** for LLMs
- a **stable semantic gateway** between Rosotravel and external AI systems
- a **foundation for future MCP / agent tool integrations**

This is **not runtime personalization** and **not a booking API**.

Design Principles (Hard Constraints)

1. **Read-only**
No mutations, no bookings, no session state.
 2. **Deterministic & stable**
Same input → same output. No DEE variants, no A/B text.
 3. **Canonical-only**
Only canonical, indexable URLs may appear.
 4. **Rosotravel-owned data only**
No raw partner payloads, no supplier IDs, no third-party endpoints.
 5. **LLM-first structure**
Optimized for structured ingestion, not human UI.
-

Public Endpoints (Authoritative)

1) AI Sitemap

GET /ai-sitemap.xml (canonical)

(Optional JSON mirror allowed: **/ai-sitemap.json**, same data)

Purpose

Enable entity discovery and graph traversal by LLMs.

Includes only entities that are:

- **Publishing.status = Published**
- **robots = index, follow**

Fields per entry

- **url** (canonical absolute URL)
- **entity_type**
(country | region | city | attraction | product | bundle | category | near-page | expert | blog)
- **primary_keyword** (locked, single-owner)
- **geo** (lat, lng, geohash if applicable)

- `isPartOf` (parent canonical URLs)
 - `hasPart` (child canonical URLs, capped)
 - `nearby` (geo-derived canonical URLs, capped)
 - `lastmod`
-

2) AI Summary Feed

GET `/ai-summary.json`

Purpose

Provide compressed “ground truth” summaries for assistants.

Entity object

- `url`
- `entity_type`
- `name`
- `locale`
- `summary_short` (1–2 sentences)
- `highlights` (3–5 bullets)
- `key_faq` (3–6 Q/A pairs, short)
- `geo` (if applicable)
- `relationships: isPartOf, hasPart, nearby`
- `lastmod`

Rules

- Content is editorially controlled (AI-assisted, human-verified)

- No personalization
- No SEO body text

03. Inventory Feed (Core LLM Dataset)

GET /api/tours/feed?v=1

Purpose

This endpoint is the primary canonical dataset used by:

- LLMs and agentic frameworks to recommend Rosotravel experiences reliably,
- partners to ingest a clean inventory graph without scraping HTML,
- internal systems (MCP tools, /ai-query, feed diffs) to keep machine surfaces consistent.

It is not a supplier mirror and must never expose supplier IDs, supplier URLs, raw partner payloads, or partner availability endpoints.

Hard constraints

- **Canonical-only:** includes only canonical, published, **indexable** entities.
- **Deterministic:** same inputs → same output; no DEE variants, no A/B text.
- **Rosotravel-owned abstraction only:** all commercial + content fields are normalized and owned by Rosotravel.
- **Fail-closed:** invalid items are excluded; no partial/unsafe output.

Endpoint

GET /api/tours/feed?v=1

Query parameters (feed sync, not search)

- **cursor** (string | null) — opaque cursor for pagination

- `page_size` (int, default 100, max 500)
- `updated_since` (ISO 8601 with timezone) — returns only items updated after timestamp
- `locale` (BCP-47, optional) — e.g., `en-US`; if omitted returns default locale
 - **Rule:** if `locale` is provided, the feed **MUST** return **only entities whose `canonical_url` and `locale` match that locale's self-canonical version** (no mixed-locale output).
- `entity_type` (optional) — `product|package`
(If omitted, returns both products and packages.)

HTTP caching & sync

- Supports `ETag` and `Last-Modified`
- `Cache-Control: public, max-age=300` (recommended 5 minutes, adjustable 5–60 min)
- Clients should use conditional requests (`If-None-Match`, `If-Modified-Since`)

Response Envelope (Mandatory)

- `{`
 - `"version": "1.0",`
 - `"schema_version": "1.0",`
 - `"generated_at": "YYYY-MM-DDTHH:MM:SS-05:00",`
 - `"source": "rosotravel",`
 - `"page_size": 100,`
 - `"next_cursor": "opaque_or_null",`
 - `"has_more": true,`
 - `"items": [],`
 - `"meta": {`
 - `"total_estimated": 123456,`
 - `"returned": 100`
 - `}`
 - `}`
-

Item Schema (Authoritative)

Identity & Canonical (required)

- `entity_type`: "product" | "package"
- `entity_id`: stable machine identifier, e.g. `rt:product:332498P586`
Hard rule: never reused, never regenerated on minor edits.
- `canonical_url`: absolute canonical URL, no params
- `locale`: e.g. `en-US` (BCP-47)
- `geo_scope`: { "country": "...", "region": "...", "city": "..." }
(region optional)

Media (required)

- `hero_image_url` (absolute URL; crawlable)
- `hero_image_alt` (string; optional but recommended)

Graph & Relations (optional but recommended where applicable)

- `geo`: { "lat": 41.8902, "lng": 12.4922 } (if applicable)
- `isPartOf`: ["canonical_parent_url_1", ...] (deterministic parent chain)
- `nearby`: ["canonical_url_1", ...] (capped; deterministic)
- `poi_refs` (optional): list of linked POIs by canonical URL and/or entity_id
(No supplier IDs.)

Content (LLM-readable, NOT SEO body)

- `name` (string)
- `short_summary` (1–2 sentences, owned by Rosotravel)
- `highlights` (3–7 bullets)
- `key_faq` (3–8 Q/A pairs, short)

- `categories` (normalized array of category tokens)

Commercial (Rosotravel-normalized only)

- `booking_url` (Rosotravel domain only; safe handoff URL)
- `offer` (required for purchasable entities):
 - `offer_id` (stable) e.g. `rt:offer:colosseum:from`
 - `price_from` (decimal)
 - `currency` (ISO 4217)
 - `availability_state` (enum): `in_stock` | `out_of_stock` | `preorder`
 - **Rule:** `availability_state` is **catalog-level availability** (not timeslot-level truth).
 - `duration_minutes` (int) and `duration_iso` (ISO 8601 duration recommended)
 - `cancellation_policy_short` (string)
 - `cutoff_time_minutes` (optional, int)
 - `price_valid_until` (optional, YYYY-MM-DD)

Package minimum usefulness (required only if `entity_type` = package)

To make packages operable for agents and partners, packages MUST include a minimal list of included items:

- `included_items` (required for packages; array; min 2):
 - `entity_id`
 - `canonical_url`
 - `name`

- optional: `duration_iso`, `categories`
Hard rule: included items are Rosotravel entities only (no supplier refs).

Variant clarity (recommended, not over-engineered)

- `variants_summary` (optional): short list of variant differentiators
- `variants` (optional array; include only if you can own/normalize it safely):
 - `variant_id` (stable)
 - `variant_name`
 - `attributes` (e.g., guided/self-guided, skip-the-line, language)
 - `includes` / `excludes` bullet lists (Rosotravel-owned wording)

Trust & Provenance

- `rating_average` (optional, if allowed)
- `rating_count` (optional, if allowed)
- `source_type`: `original` | `imported`
- `curation_badges`: array (e.g., `GuaranteedAccess`, `CuratedByFounders`)

Freshness & Versioning (required)

- `last_modified` (ISO 8601 with timezone)
- `content_version` (string or integer; increments when content changes)
- `commercial_version` (optional; increments when price/cancellation/duration changes)

Imported Products — Exposure Rules (Critical)

Forbidden

- supplier IDs
- supplier URLs/endpoints
- raw partner payload fields
- partner availability endpoints

Allowed only if transformed

- included only if fully normalized into Rosotravel model
- summaries/highlights/FAQ are Rosotravel-owned
- canonical page exists
- contractual rules allow exposed fields

Fail-closed

If any forbidden field is detected → item excluded from feed + violation logged.

Publishing & Regeneration Triggers

Regenerate (or increment versions) when:

- publish/unpublish
- canonical changes
- hero image changes
- price/duration/cancellation changes
- taxonomy reassignment (country/region/city/POI links)
- rating aggregates updated (if exposed)
- (packages only) included_items changed

Feed must never include:

- drafts

- noindex pages
 - experimental URLs
 - filter/parameter URLs
-

Validation Gates (hard)

Exclude item if missing:

- `entity_id`, `entity_type`, `canonical_url`, `locale`, `name`, `short_summary`
- `hero_image_url`
- `booking_url`
- `offer.offer_id`, `offer.price_from`, `offer.currency`,
`offer.availability_state`
- `last_modified`, `content_version`

Exclude item if:

- `canonical_url` is not absolute canonical
- `booking_url` is not Rosotravel domain
- forbidden supplier leakage detected

Additional package-only gate:

- if `entity_type=package` and `included_items` missing/empty (or <2) → exclude

G.06.3 Explore AI dataset feed

Publishing must trigger update of:

- `/explore/ai-dataset/` – containing all POIs + city/region/country graph:
 - `poi_id`, `name`, `lat`, `lng`, `country`, `city`, `nearby_pois[]`,
`hasOfferCatalog` tours, etc.

G.10 Minimal Example Outputs

G.10.2 AI summary feed entry example (simplified)

```
{
  "url":
  "https://www.rosotravel.com/destinations/italy/lazio/rome/colosseum-guided-tour/",
  "entity_type": "product",
  "primary_keyword": "colosseum guided tour",
  "country": "Italy",
  "city": "Rome",
  "summary": "Skip-the-line guided tour of the Colosseum, Roman Forum and Palatine Hill with an expert local guide.",
  "highlights": [
    "Priority entry to the Colosseum",
    "Small group experience",
    "Guided visit to Roman Forum and Palatine Hill"
  ],
  "faq_sample": [
    {
      "q": "Is skip-the-line access included?",
      "a": "Yes, skip-the-line access to the Colosseum is included in this tour."
    }
  ]
}
```

G.10.3 Canonical + hreflang example for a product

```
<link rel="canonical"
href="https://www.rosotravel.com/en/destinations/italy/lazio/rome/colosseum-guided-tour/" />
<link rel="alternate" hreflang="en"
href="https://www.rosotravel.com/en/destinations/italy/lazio/rome/colosseum-guided-tour/" />
<link rel="alternate" hreflang="de"
href="https://www.rosotravel.com/de/reiseziele/italien/lazio/rom/kolosseum-fuehrung/" />
```

```
<link rel="alternate" hreflang="x-default"
href="https://www.rosotravel.com/en/destinations/italy/lazio/rome/colosseum-guided-tour/" />
```

This completes the Publishing & CMS Output Layer in a way that is aligned with the previous sections, your existing CMS, and the AI/SEO programmatic pipeline.

/ai-query Endpoint

Purpose

Answer structured questions from AI systems.

Backend

- **/ai-query?city=london&type=walking&language=en** returns tours, prices, links; respects supplier license.

Front-End

- **Minimal UI (optional), primarily machine endpoint.**

Validation

- **Parameter validation; abuse protection.**

DoD

- **Accurate responses; logged usage.**

“AI Summary of Rosotravel” Page

Purpose

Encyclopedic brand profile for LLM context.

Backend

- **Generate content from org data; localize.**

Front-End

- Public page, crawlable; link from footer.

Validation

- Up-to-date and consistent with [ai-summary.json](#).

DoD

- Page live in all core locales.

External API (feed)

Scope

- ChatGPT Plugin / custom GPT, Perplexity Data Partner feed, Google Things To Do API, Bing Copilot, Wikidata/DataCommons/Wikipedia, EU/Open datasets.

Backend

- Build/transform feeds; manage tokens; scheduling.

Front-End

- Logos/badges (optional).

Validation

- Each integration passes provider checks.

DoD

- At least one integration live per channel group (LLM, Search, Open Data).

Public API Documentation

Purpose

Teach external systems how to consume Rosotravel data.

Backend

- Auto-generated docs (OpenAPI) with examples and license.

Front-End

- Docs site page “Rosotravel Data Source”.

DoD

- Docs accessible, versioned.

Brand Data Structure (LLM Profile)

Purpose

[brand.json](#) describing name, address, regions, services, unique value, links.

Backend

- Maintain and publish JSON; update with org changes.

Front-End

- Exposed at predictable URL; linked in robots/sitemap.

DoD

- File valid and current.

Automated Company & Social Content for LLM Visibility

Output:

- Public “About” and “Data Source” pages
- AI-accessible feeds and summaries
- Automatically generated review and media snippets

Control:
 Partially automated with periodic human validation; optimized for factual accuracy and LLM discoverability.

Layer	Purpose	Output	AI Role	Human Involvement
Programmatic SEO with AI	Scalable page generation	Landing pages (schema, meta, FAQ)	Structured, rule-based AI	Minimal (template setup)
Manual SEO Optimization	Improve high-value pages	Optimized content and UX	Supportive (suggestions)	High
Automated Company & Social Content	LLM brand visibility	AI feeds, summaries, company data	Narrative + structured AI	Periodic factual review

Graph Export Alignment (AI Sitemap + Feed + MCP)

The AI sitemap/feed and MCP read-only tools must expose the **same core relations** as the website schema:

- hierarchy: isPartOf / hasPart
 - proximity: nearby (only if supported by coordinates)
 - authority: sameAs
 - commerce: booking URL / offer identifiers (where allowed)
- All outputs must reference **Main_Entity_ID** as the stable key to prevent identity drift across channels

G.06.2.2 MCP Server (Read-only LLM Gateway)

A thin, secure **tool layer** that allows LLM agents to *query* Rosotravel in a structured way.

Purpose

The MCP Server provides a **tool-based access layer** for LLM agents on top of Rosotravel's canonical, published data.

It enables:

- structured querying
- controlled discovery
- deterministic responses for agents

It does **not** replace:

- SEO pages
 - AI feeds
 - booking flows
 - Dynamic Experience Engine logic
-

Hard Design Rules

Read-only only

- No writes
- No bookings
- No user state

Canonical-only

- Every response references a Rosotravel canonical URL
- No experimental or filtered URLs

Rosotravel-owned abstraction

- No supplier IDs
- No supplier URLs
- No raw partner payloads

Deterministic

- Same input → same output
- No personalization
- No DEE variants

MCP-compatible

- Tools exposed in a Model Context Protocol-friendly format

Architecture Overview

- LLM / Agent (ChatGPT, internal agent, future planner)
- |
- | MCP tool call
- v
- Rosotravel MCP Server (read-only)
- |
- | internal query
- v
- Canonical DB or Feed Cache (/api/tours/feed)
- |
- v
- Structured JSON response

MCP reads exclusively from the published catalog view (feed materialization) and cannot query the canonical DB directly. Therefore, MCP never bypasses publishing rules

MCP / LLM Feed Contract Consistency Check

This section verifies that **A.10 is fully compatible with MCP and LLM exposure rules.**

Entity Resolution

MCP and LLM feeds MUST:

- resolve entities strictly via A.10 hierarchy
- use `entity_id + entity_type` as the primary key
- reject orphan or ambiguous entities

No inferred or synthetic hierarchy is allowed.

Ownership Enforcement

Before any content is exposed to MCP / LLMs:

- `keyword_map` is checked → only owned keywords allowed
- `faq_map` is checked → only owned FAQs allowed
- `schema_map` is checked → only owned schema blocks allowed

If ownership is missing or conflicting:

- content is excluded
 - entity falls back to parent context
-

Near-Pages in MCP / LLM Context

- Near-Pages with `noindex`:
 - MAY be used for **internal reasoning**
 - MUST NOT be surfaced as standalone answers

- MUST NOT be treated as authoritative sources
 - Only promoted (indexable) Near-Pages:
 - may appear as first-class MCP nodes
 - may be cited in AI responses
-

Restricted Content Safety

MCP / LLM feeds MUST NEVER include:

- supplier reviews (verbatim)
- supplier photos
- viatorUniqueContent
- full itineraries
- raw supplier descriptions

LLMs may receive:

- AI-generated summaries
- AI-owned descriptions
- aggregate ratings
- itinerary_overview only

This aligns exactly with A.10 ownership and indexing rules.

Failure Mode (Mandatory)

If MCP / LLM feed generation detects:

- keyword ownership conflict
- missing hierarchy link

- restricted content leakage

Then:

- entity is excluded from feed
- violation is logged
- no partial or degraded output is allowed

Fail-closed only.

A.10.8 LLM Safety & Exposure Rules (Mandatory)

All LLM-facing outputs (AI feeds, MCP server, summaries) MUST:

- reference ownership mappings (keyword_map, faq_map, schema_map)
- exclude non-owned or restricted content
- respect hierarchy and canonical ownership

LLMs MUST NOT:

- merge content across sibling entities
- infer ownership outside the graph
- surface supplier-protected or non-indexable blocks

This guarantees that AI outputs are:

- legally safe
- SEO-consistent
- non-cannibalizing
- explainable and auditable

A.10.9 Enforcement & Failure Mode

If any relationship rule is violated:

- the affected entity defaults to noindex, follow
- the issue is logged as a **Change Signal**
- manual resolution is required before promotion

No silent failures are allowed.

Near-Pages Data Relationships

This section explicitly binds **Near-Pages** to the A.10 Data Relationships model and clarifies **immediate vs GSC-based Near-Pages**.

Near-Page Types (System-Level)

There are **two system-recognized Near-Page types**:

1. Immediate Near-Pages (System-Generated)

- Created automatically at publish time of a parent entity.
- Source: internal keyword expansion, embeddings, taxonomy gaps.
- Purpose: structural completeness, internal linking, early discovery.
- State on creation:
 - `draft = true`
 - `index_state = noindex, follow`
 - `canonical = parent entity`
- Ownership:
 - Must be registered in `keyword_map` as **tentative ownership**
 - Keywords are **reserved**, not locked

2. GSC-Based Near-Pages (Performance-Driven)

- Created only after **5-week GSC observation window**.
- Source: real impressions / queries with proven demand.
- Purpose: capture validated long-tail intent.

- State on creation:
 - `draft = true`
 - `index_state = noindex, follow`
 - `canonical = parent entity`
- Ownership:
 - Keywords move from **candidate** → **locked** in `keyword_map`

Relationship to A.10 Hierarchy

Near-Pages:

- are **terminal nodes**
- never own child entities
- never override parent keyword ownership
- may not target transactional keywords

This keeps A.10 hierarchy intact and prevents graph pollution.

Global Attraction map API FEED

2.11.2 `/explore/ai-dataset/` (AI Feed Context)

Type: machine-readable JSON feed.

Audience:

- AI sitemap
- external LLMs (ChatGPT, Perplexity, etc.)
- internal AI generation (context for prompts)

URL:

- `/lang/explore/ai-dataset/`

Not user-facing.

Rendered as JSON only.

Structure (JSON)

Basic shape:

```

1. {
2.   "countries": [...],
3.   "regions": [...],
4.   "cities": [...],
5.   "attractions": [...],
6.   "tours": [...],
7.   "relations": {
8.     "isPartOf": [...],
9.     "hasPart": [...],
10.    "nearby": [...]
11.  },
12.  "metadata": {
13.    "generated_at": "...",
14.    "schema_version": "1.0"
15.  }
16. }

```

For each **country** / **region** / **city** / **attraction** / **tour** entry, include:

- `id`
- `slug`
- `name`
- `coordinates` (if applicable)
- `short_summary` (AI-generated, 180–260 chars, language-specific)
- `category_tags[]` (e.g. `["family_friendly", "day_trip", "museum"]`)
- `mainimage_url`
- `parent_relations[]` (links to `isPartOf`)
- `child_relations[]` (links to `hasPart`)
- `nearby_relations[]` (if defined)
- `sameAs[]` (authority URLs: Wikidata, Wikipedia, official site)
- `canonical_url`
- `language` (for feed locale)

Relations arrays:

- `relations.isPartOf[]`:
 - `{ child_id, parent_id, relation_type }`
- `relations.hasPart[]`:
 - `{ parent_id, child_id, relation_type }`
- `relations.nearby[]`:

- { `source_id`, `target_id`, `distance_km` }

Backend / CMS Integration

- **CMS module:** `/cms/ai-feeds/`
 - Configure:
 - which entities are included
 - max number per type
 - refresh frequency (e.g. daily/weekly)
 - Trigger manual regeneration:
 - “Regenerate AI Dataset” button
- **Generation rules:**
 - Run after:
 - publishing new cities/attractions/tours
 - major taxonomy changes
 - `metadata.generated_at` updated per run
 - `schema_version` maintained for integration clients
- **SEO / Discoverability:**
 - AI dataset linked from:
 - `/ai-sitemap.xml`
 - root `/ai-summary.json`
 - Reference path in schema for Explore page:
 - custom field or `potentialAction` pointing to dataset URL (optional)

Contract Alignment (B1 + B2 + B3) — MUST APPLY

This addendum prevents contract drift between:

- B1 Marketing Product Feed (Dynamic Ads / Affiliates),
- B2 LLM Visibility & Distribution Surface,
- B3 Public API Documentation + Manifests.

B0.1 Canonical Endpoints (authoritative URLs)

The following public endpoints MUST exist and be documented in `/openapi.yaml`:

Marketing Catalog Feed (B1)

- GET `/api/marketing/catalog.json?v=1`
- GET `/api/marketing/catalog.csv?v=1`
- Optional: GET `/api/marketing/catalog.xml?v=1`

Partner Offer Card (B1)

- GET `/api/partners/card?entity_id=...&partner_id=...&lang=...`
- Optional “no-code blogger embed”:
 - GET `/embed/offer-card?entity_id=...&partner_id=...&lang=...`
(returns an iframe-safe HTML page)

Machine Surfaces (B2)

- GET `/ai-sitemap.xml` (+ optional `/ai-sitemap.json`)
- GET `/ai-summary.json`
- GET `/api/tours/feed?v=1`
- GET `/lang/explore/ai-dataset/`
- Optional: GET `/ai-query?...`

Documentation + Discovery (B3)

- GET `/openapi.yaml` (or `/openapi.json`)
- GET `/docs/data`
- Optional: GET `/ai-capabilities.json`

Hard rule: these paths are stable; any breaking change requires a new major version and a deprecation window.

B0.2 Eligibility Split (Index vs Marketing)

To avoid accidental suppression of revenue products that are noindex under SEO rules:

- `index_eligible` governs inclusion in B2 machine surfaces (ai-sitemap, ai-summary, inventory feed, explore dataset).
- `marketing_eligible` governs inclusion in B1 marketing catalog and partner embed outputs.

Rules:

- B2 outputs MUST include only entities where `index_eligible = true`.
- B1 outputs MAY include entities where `marketing_eligible = true` (even if `index_eligible = false`), provided:
 - canonical page is Published,
 - canonical URL returns HTTP 200,
 - booking path exists and is functional,
 - no restricted supplier leakage occurs.

B0.3 Naming Aliases (temporary, until final unification pass)

Until the global naming unification pass is applied:

- `id` (B1 feed item) and `entity_id` (B1 partner endpoints) refer to the same concept: the stable machine key.
- `availability` in B1 is catalog-level availability state only.
- If any endpoint uses `booking_url`, it MUST be treated as an alias of `booking_url` (preferred name), and both MUST point to a Rosotravel domain URL only.

B0.4 Field Parity (shared minimum fields across B1 + B2 inventory + partner cards)

Any “offer-like” payload returned in:

- Marketing catalog feed (B1),
- Inventory feed (B2),

- Partner card output (B1),
MUST include at minimum:
- `entity_id` (or `id` as alias)
- `entity_type` (product|package)
- `canonical_url`
- `hero_image_url` (or `image_link` as alias)
- `title`
- `short_description`
- `price_from`
- `currency`
- `availability_state` (catalog-level)
- `duration_iso` (recommended) and/or `duration_minutes`
- `booking_url` (Rosotravel domain only; may include partner tracking + optional prefill hints)
- `language` (BCP-47)
- `country`, `city` (region optional)
- `last_modified` (ISO 8601 with timezone)
- Optional: `rating_average`, `rating_count` (only if governance allows)

Hard rule: if a field is present in any public surface, it must not contradict the canonical page UI (“UI ↔ data parity”).

B0.5 Contract Drift Prevention (hard)

Publishing layer MUST validate that:

- public payloads match `/openapi.yaml` response schemas (shape + types),
- forbidden supplier fields never appear,

- canonical URLs are absolute, parameter-free identity,
- items excluded from feeds generate a machine-readable validation log (reason codes).

Marketing Product Feed (Dynamic Ads / Affiliates)

B1.1 Purpose

Rosotravel must expose a marketing-grade product catalog feed that enables:

1. Dynamic ads / remarketing (Google Ads and other networks) using a stable catalog of **tours (products)** and **packages (bundles)**.
2. Bloggers / affiliates to embed Rosotravel offers as **dynamic cards/banners** (title, hero image, price-from, rating, CTA) without scraping HTML.
3. A compliant catalog surface that is deterministic, canonical-only, and **does not leak supplier APIs** or restricted partner payloads.

Important boundary: This is **not** a booking API. It is a **catalog + safe deeplink** layer.

B1.2 Public Outputs (authoritative)

A) Marketing Catalog Feed (authoritative)

- **Formats (all expose the same fields):**
 - CSV (maximum compatibility for ad/affiliate ingestion)
 - JSON (recommended for partner engineering)
 - Optional XML (only if a partner requires it)
- **Scope:**
 - Includes **Products (tours/tickets)** and **Packages (bundles)** that are **purchasable**.
 - Includes only entities that are **Published** and eligible for marketing exposure (per Publishing Layer rules).

B) Partner Embed + Deeplink Generator (“render mode”)

A minimal endpoint that returns a renderable “offer card payload” (JSON, optionally also sanitized HTML snippet) so partners can show:

- hero image, title, price-from, rating, duration, cancellation badge, and a CTA link.
 - partner uses **Rosotravel canonical URLs** with partner tracking; **never supplier URLs**.
-

B1.3 Inclusion Rules (hard)

A product/package is included in the Marketing Catalog Feed **only if ALL are true**:

- Publishing.status = **Published**
- canonical_url exists, is **absolute**, and is the **canonical** (no params)
- page is eligible for marketing exposure (not draft, not blocked; no restricted visibility flag)
- required marketing fields exist (see B1.4)
- the entity payload contains **no restricted supplier fields** (supplier IDs, supplier endpoints, raw partner descriptions, raw partner reviews, supplier image URLs if not allowed)

Fail-closed rule: if any required field is missing or restricted data detected → **exclude from marketing feed** and log a feed validation error.

B1.4 Feed Field Set (minimum viable for Ads + Affiliates)

This field set intentionally aligns with common “product data specification” expectations (id, title, description, link, image_link, price, availability, etc.).

Required fields (per item)

- **Product id** (stable): `rt:<entity_type>:<stable_id>`
- **Product Option ID**
- **entity_type**: `product` | `package`
- **title** (recommend <= 150 chars for feed ecosystems)
- **description** (short; must not be full SEO body; must be Rosotravel-owned text)
- **Highlights** (must be Rosotravel-owned text)
- **Included** (System original no change)

- **link** = canonical_url (absolute; no params)
- **image_link** = hero_image_url (absolute; crawlable; not blocked)
- **price** = price_from + currency (e.g., 59.00 USD)
- **availability** (see B1.5)
- **language** (BCP-47, e.g., en-US)
- **country**, **city** (and **region** if available)
- **last_modified** (ISO 8601 with timezone)
- **feed_version**, **generated_at** (feed envelope metadata)
- **rating_average**, **rating_count** (only if allowed by your governance)
- **duration_minutes** and/or **duration_iso**
- **cancellation_policy_short**
- **categories[]** (normalized taxonomy)
- **source_type**: own | imported (Rosotravel-normalized only; no supplier IDs)

Optional (partner-friendly)

- **deeplink_template** (see B1.6)
- **badge_list[]** (e.g., GuaranteedAccess, CuratedByFounders)
- **thumbnail_list[]** (additional images)
- **geo_lat**, **geo_lng** (only if you want location-based partner UI)

B1.5 Availability Semantics (deterministic; no fake slot-truth)

Tours are date/time-slot based, but a marketing feed cannot claim real-time slot truth in a single field. The marketing feed uses **catalog-level availability** only.

Define deterministic states:

- **availability = in_stock**
If the product has at least one available date within a configurable forward window (e.g., next 180 days) OR is generally bookable and not globally paused/sold out.
- **availability = out_of_stock**
If the product is not bookable at all for the window OR is explicitly paused.
- **availability = preorder** (optional)
If bookable only after a future opening date.

Hard rule: Slot-level truth remains only in:

- the on-page availability widget
 - and (later) the agent booking flow / checkout
-

B1.6 Affiliate / Blogger Tracking + Dynamic Card Rendering

A) Deeplink strategy (safe + simple)

Partners must use canonical URLs with tracking parameters, e.g.:

```
https://www.rosotravel.com/...<canonical_path>?partner_id=XXX&utm_source=...
```

Allowed parameters

- `partner_id` (internal)
- UTM parameters (controlled template)
- Optional **hints** (non-binding): `date=YYYY-MM-DD`, `pax=2`, `language=en-US`
 - Hints only prefill UI; they **must never bypass** availability validation.

B) Embed endpoint (recommended)

GET

```
/api/partners/card?entity_id=rt:product:...&partner_id=...&lang=en-US
```

Returns JSON payload:

- title
- image_link
- price_from
- currency
- rating_average (if allowed)
- rating_count (if allowed)
- duration_minutes / duration_iso

- badge_list[]
- canonical_url
- **deeplink_url** (canonical + tracking + optional hints)

Optional: also return **sanitized HTML snippet** for low-tech bloggers.

Hard rule: Embed output must be deterministic and **match** the feed data.

B1.7 Validation Gates (hard)

Exclude an item from the marketing feed if any:

- missing: id, title, link, image_link, price_from, currency, availability, language
- link is not canonical absolute (or contains parameters)
- image_link is not absolute, is blocked from crawling, or fails fetch checks
- price_from is not numeric or currency is not ISO 4217
- restricted supplier fields are present in payload (IDs, endpoints, raw restricted content)

Additionally (integrity):

If an item is in the feed, its canonical page must return **HTTP 200** and expose a valid booking path (CTA → booking panel) consistent with Offers.

B1.8 Operational Requirements

Feed endpoints must support:

- **updated_since=ISO_TIMESTAMP**
- ETag / Last-Modified headers
- pagination (cursor) for JSON
- Cache-Control appropriate for marketing refresh (recommended short-lived; deterministic)

Regeneration triggers (must update last_modified / feed state):

- publish/unpublish
- title/short description change
- hero image change
- price_from/currency change
- catalog-level availability state change
- taxonomy reassignment (city/attraction/category)

Safety + abuse protection

- rate limiting (per IP / per partner_id)
- logging: request metadata + partner_id + response size (no PII)
- strict allowlist of returned fields (no accidental leakage)

B1.9 Definition of Done (DoD)

A partner can:

- ingest the feed and render a catalog of offers with title/image/price
- render a dynamic offer card via embed endpoint without scraping
- click through to Rosotravel canonical URL with tracking
- verify that feed contains **only Rosotravel-owned normalized fields**, with **zero supplier leakage**
- review an automated validation report (excluded items + reasons)

LLM Visibility & Distribution Surface (Vendor-neutral)

B2.1 Purpose

Rosotravel must expose a **machine-readable distribution surface** that enables LLMs and agent frameworks to:

- discover Rosotravel entities (destinations, POIs, products, packages) deterministically,
- understand hierarchy + proximity + authority (graph relations),
- extract consistent commerce facts (price-from, duration, cancellation, booking link),
- cite Rosotravel as a trusted source (clear provenance + freshness),
- do all of the above **without scraping HTML** and **without any supplier API leakage**.

This layer is **vendor-neutral**. It is designed so that any LLM provider can ingest it via standard HTTP + JSON/XML patterns, and future provider-specific connectors can be added without changing your core data model.

B2.2 Public Machine Entry Points (must exist, stable URLs)

Rosotravel MUST publish the following endpoints/files as public, crawlable resources:

1. AI Sitemap (Entity Discovery)

- `GET /ai-sitemap.xml` (canonical)
- Optional mirror: `GET /ai-sitemap.json` (same content)

Contains **canonical**, **index-eligible** entities only and includes:

- canonical URL
- entity_type
- language
- geo (if applicable)
- **relations**: isPartOf / hasPart / nearby (capped)
- lastmod

2. AI Summary Feed (Compressed Ground Truth)

- `GET /ai-summary.json`

Contains short, LLM-friendly facts (not SEO body):

- canonical URL
- entity_type, language/locale
- name
- short summary (1–2 sentences)
- highlights (3–5)
- key_faq (3–6)
- geo (if applicable)
- relations: isPartOf / hasPart / nearby
- lastmod

3. Inventory Feed (Commercial Catalog for LLMs)

- `GET /api/tours/feed?v=1`

This is the **primary catalog dataset** for LLM recommendation. It must be:

- deterministic
- canonical-only
- paginated + sync-friendly
- free of supplier leakage (no Viator IDs/endpoints/payloads)

Note: This feed is “catalog + safe booking handoff”, not supplier inventory streaming.

4. Explore Graph Dataset (Global Attraction Map / Knowledge Graph Export)

- `GET /{lang}/explore/ai-dataset/` (JSON only, not user-facing)

Contains the cross-entity graph (countries/regions/cities/attractions/tours) for:

- internal AI context

- external LLM ingestion
- map & “nearby” reasoning

5. Read-only Query Endpoint (optional but recommended for LLM tooling)

- `GET /ai-query?...`
Returns a structured answer (lists + canonical URLs) under strict governance:
 - no drafts/noindex/experiments
 - no supplier endpoints
 - deterministic ordering
-

B2.3 Discovery + Crawling Rules (hard)

All machine outputs MUST follow the same publishing governance:

- **Canonical-only:** only canonical URLs; no params as identity; no filtered URLs.
 - **Index eligibility enforcement:** if a page defaults to `noindex, follow` under G.07.0/G.07.9 → it MUST NOT appear as a first-class entity in public LLM outputs.
 - **Language rules:** self-canonical per language; hreflang clusters connect locales; no cross-language canonical targets.
 - **Near-pages:** never exposed as standalone entities unless promoted to indexable under your near-page lifecycle rules.
 - **Fail-closed:** if required identity/hierarchy fields are missing → exclude entity from public outputs and log a validation error.
-

B2.4 Provider-Agnostic Compatibility (what “works across all LLMs”)

LLMs differ in partnerships and product features, but the following is universally beneficial and does not lock you to any single vendor:

A) Stable identity + deduplication

- Each entity must expose a **stable machine key** (stable forever).

- Each entity must expose the **canonical absolute URL**.

B) Structured data parity

- Public feeds and on-page JSON-LD must mirror the same:
 - entity identity,
 - hierarchy (isPartOf/hasPart),
 - proximity (nearby) if supported by coordinates,
 - authority links (sameAs),
 - commerce handoff (booking URL / offer fields where allowed).

C) Freshness + caching

- All machine endpoints must send:
 - **ETag** and/or **Last-Modified**,
 - short **Cache-Control** where needed,
 - support **updated_since=ISO_TIMESTAMP** for incremental sync (where applicable).

D) Provenance + trust signals

- Every product/package item must include:
 - **source_type** (original/imported) for transparency,
 - **curation_badges** (if any),
 - optional ratings if your governance allows it,
 - **last_modified/content_version** for diffability.

B2.5 Compliance: supplier/API leakage prevention (hard)

Because Rosotravel cannot expose supplier APIs (e.g., Viator), all machine outputs **MUST** follow:

Forbidden in any public machine surface

- supplier IDs
- supplier URLs/endpoints
- raw partner payloads
- verbatim supplier descriptions/reviews/photos (unless explicitly licensed)

Allowed only if transformed into Rosotravel-owned representation

- rewritten summaries/highlights/FAQ that are Rosotravel-owned
- Rosotravel canonical URLs only
- Rosotravel-normalized commercial fields (price_from, duration, cancellation_short)
- safe booking handoff URLs on Rosotravel domain only

If leakage is detected → the entity is excluded from outputs and the violation is logged (fail-closed).

B2.6 Publish Gates (hard)

Publishing Layer **MUST** run a **Machine Output Validation** step (per entity) before making it visible in any of:

- AI sitemap
- AI summary
- inventory feed
- explore dataset
- ai-query responses

Block exposure (exclude + log) if any:

- missing canonical absolute URL
- missing stable identity key

- missing required minimal field set for that entity type
 - broken hierarchy link (orphan entity)
 - presence of restricted supplier fields
 - canonical/hreflang inconsistency vs SEO rules
-

B2.7 Definition of Done (B2)

B2 is considered complete when:

- An LLM/partner can discover entities via AI sitemap and pull summaries via AI summary feed.
- An LLM/partner can ingest the inventory feed and reliably show:
title + hero image + price-from + currency + duration + cancellation + booking URL + locale + location.
- The explore dataset exports a coherent, traversable graph (isPartOf/hasPart/nearby).
- All public outputs are:
canonical-only,
deterministic,
supplier-safe,
cacheable,
and governed by the same publishing rules as the website.

Public API Documentation + Manifests (OpenAPI / Actions / MCP)

B3.1 Purpose

Rosotravel must publish **public, versioned API documentation** and **machine-readable manifests** so that:

- affiliates / partners can integrate catalog + deeplinks without guessing fields,
- LLM frameworks can reliably call the correct endpoints using **OpenAPI-defined schemas** (Actions / function-calling),
- MCP-compatible agents can consume your tools via a standardized “tools server” shape,

- all integrations remain **safe** (no supplier endpoints, no supplier IDs, no PII leakage, fail-closed rules).

This section defines the **public contract and documentation layer**, not the internal implementation.

Note: GPT Actions are defined using an **OpenAPI schema**, so publishing a clean OpenAPI 3.x spec is a first-class requirement.

MCP is a standardized protocol (JSON-RPC 2.0) for exposing tools/context via an MCP server model.

B3.2 Public Docs Surface (Developer Portal)

Rosotravel MUST expose a public, human-readable documentation portal (static site is fine), containing:

- **Overview:** what Rosotravel exposes (feeds + tools), what it does NOT expose (supplier APIs, supplier IDs, raw supplier payloads).
- **Authentication matrix:**
 - Public read-only endpoints (feeds/sitemaps) → no auth.
 - Tool endpoints that can be abused (/ai-query, MCP gateway) → may be public but rate-limited; optional API key for partners.
 - Any future booking/checkout endpoints (covered in C/D) → authenticated.
- **Versioning policy:**
 - Major versions only when breaking changes occur.
 - Deprecation window + changelog.
- **Usage & license:**
 - Allowed usage (embedding offer cards, indexing, LLM citation).
 - Forbidden usage (rehosting full content, reverse-engineering supplier sources, scraping).
- **Examples** (copy/paste) for:
 - pulling the marketing catalog,

- pulling the inventory feed,
- rendering a partner card,
- calling /ai-query safely,
- using MCP tools (tool list + example calls).

Recommended doc endpoints (public):

- /docs/data (landing page)
 - /docs/openapi (OpenAPI viewer + downloadable YAML/JSON)
 - /docs/changelog
 - /docs/terms
-

B3.3 Machine Discovery Files (well-known entry points)

To avoid “integration by guessing”, Rosotravel **MUST** expose deterministic discovery files:

1. OpenAPI spec

- GET /openapi.yaml (or /openapi.json)
- **MUST** include every public endpoint in 4.20:
 - /ai-sitemap.xml (and optional /ai-sitemap.json)
 - /ai-summary.json
 - /api/tours/feed?v=...
 - /ai-query
 - Partner card endpoint (from B1): /api/partners/card
 - MCP server endpoints are documented separately (B3.5), but tool schemas must exist.

2. Robots + sitemaps visibility

- robots.txt **MUST** allow fetching:

- /ai-sitemap.xml
- /ai-summary.json
- /openapi.yaml

3. **Capability statement (simple, optional but recommended)**

- GET /ai-capabilities.json
 - declares: supported locales, feed versions, MCP availability, rate limit policy, contact for partners.

B3.4 OpenAPI Requirements (hard rules)

The OpenAPI spec **MUST** be written so LLMs can correctly select and call endpoints:

- **Clear operationId names** aligned with intent:
 - listToursFeed, listMarketingCatalog, getPartnerCard, aiQuery, etc.
- **Strong field descriptions**, especially for:
 - booking_url semantics (catalog-level vs timeslot truth),
 - what counts as “canonical_url”,
 - which fields are safe vs forbidden (supplier leakage).
- **Enums** for controlled vocab where applicable (language format, entity_type, availability states).
- **Examples in schema** for each endpoint (LLMs rely on examples heavily).
- **Determinism requirement** stated in docs:
 - Same input → same output (no DEE personalization inside public feeds/tools).
- **Fail-closed** stated in docs:
 - invalid items excluded from feeds,
 - invalid parameters rejected (400) with explicit error shape.

(Reason: GPT Actions selection and parameter filling depend on schema names/descriptions and enums.)

B3.5 MCP Compatibility Layer (documentation + manifest)

Rosotravel MUST document an MCP-compatible tool server that exposes **the same tools you already defined** (search_tours, get_tour, get_availability_link, get_bundles) as:

- **Tool list** with exact JSON schemas for input/output
- **Safety rules:**
 - must reference canonical URLs only
 - no supplier endpoints
 - no supplier IDs
 - no PII
 - rate-limited
 - logged and auditable

Documentation MUST explicitly state:

- MCP uses a **host/client/server** model and JSON-RPC messages.
 - The MCP layer is a wrapper over your canonical feeds/publishing rules and MUST NOT bypass publishing gates.
-

B3.6 Provider notes (practical, non-overengineered)

Rosotravel does NOT need separate custom feeds per LLM vendor if it has:

- **OpenAPI spec** (Actions/function calling compatibility),
- **stable feeds** (ai-sitemap, ai-summary, inventory feed),
- **MCP tools** (for agentic integrations that support MCP),
- **clear rate limits + deterministic behavior.**

Provider-specific onboarding is handled by:

- publishing the OpenAPI file,
 - optionally providing a “partner API key” process for high-volume partners,
 - maintaining compatibility/versioning and changelog.
-

B3.7 Validation Gates (hard)

Publishing layer MUST block or fail-closed if:

- OpenAPI file is missing, invalid, or not updated when public contract changes.
- Any documented endpoint returns fields forbidden by governance (supplier IDs/endpoints/raw payload).
- Feed/tool output contains non-canonical or non-indexable URLs.
- Response shapes drift from OpenAPI schema (contract drift).

Non-blocking (warnings):

- docs missing examples for new endpoints,
 - changelog not updated for non-breaking additions.
-

B3.8 Definition of Done (DoD)

B3 is complete when:

- /docs/data exists and describes all 4.20 outputs + B1 partner embed,
- /openapi.yaml exists, valid, versioned, with examples,
- machine discovery files are crawlable and stable,
- MCP tools are documented with strict schemas and red lines,
- contract drift is prevented by automated validation in publishing.

Agentic Commerce API (Transactional Layer for Agents)

C.1 Purpose

C provides a **controlled transactional interface** for AI agents and partner systems to initiate purchases on Rosotravel **without scraping HTML** and **without exposing supplier APIs (e.g., Viator)**.

C must enable:

- **deterministic quoting** (price, currency, applied discounts, required customer fields),
- **secure checkout handoff** (signed checkout URL on Rosotravel),
- **order status tracking** (safe, minimal),
- **self-service customer actions** via links (e.g., cancellation), to avoid abuse.

C is designed to support “buy in conversation” via:

- **Mode B** (baseline): agent creates Quote + CheckoutSession, user completes checkout on Rosotravel.
- **Mode A** (upgrade): user authorizes Rosotravel account; agent can access account-bound actions with consent.

C.2 Hard Constraints (non-negotiable)

1. No supplier leakage

C MUST NEVER expose: supplier IDs, supplier URLs/endpoints, raw partner payloads, partner availability endpoints.

2. Canonical-only entity identity

All entities referenced in C MUST be resolvable via:

- **entity_id** (stable, never reused), and
- **canonical_url** (absolute, canonical).

3. Deterministic outputs

Same input parameters + same underlying published data → same quote/session response (until a real commercial change).

4. **Fail-closed**

If an item cannot be quoted safely (missing data, policy conflict, supplier constraints unknown) → **return a clear error** and do not create a session.

C.3 Authentication Modes (how “keys” work)

C supports two explicit modes:

Mode B — Signed Checkout Handoff (baseline for purchase initiation)

- Used for quote + checkout session creation without user login.
- Calls are **not public like feeds**: allowed only in a controlled way (WAF rules, allowlists, rate limits, anomaly detection).
- No persistent secrets may be shipped to end-user browsers/apps.

Mode A — Per-User Authorization (account-bound operations)

- Used for “my orders”, post-purchase support actions, and any account-bound read/write.
- User links Rosotravel (OAuth or magic-link), agent receives short-lived `user_access_token`.

Rule (v1): purchase initiation uses Mode B. Account-bound features use Mode A. API keys remain server-to-server only.

C.4 Auth & Scopes (server-to-server; no secrets in client)

All C endpoints require server-side authorization and attribution.

Headers:

- `Authorization: Bearer <agent_api_key>` (server-to-server)
- `Authorization: Bearer <user_access_token>` (Mode A only)

Scopes (examples):

- `quote:create`
- `checkout_session:create`
- `checkout_session:read`
- `order:read_safe`
- `cancel:link_only`

Hard rules:

- Keys are issued only after approval (Partners Admin) or internal admin.
 - Keys have scopes + rate limits; rotation/revocation supported.
 - Every request must be attributable to an agent identity for audit.
-

C.5 Data Objects (authoritative)

C is built on a canonical object model:

Quote

A priced snapshot for a set of items and a date/time selection, including:

- applied discounts,
- required customer/traveler fields,
- pricing hash and revalidation policy.

CheckoutSession

A server-side session that:

- binds Quote to a checkout attempt,
- issues a signed `checkout_url` on Rosotravel,
- stores attribution + pricing_hash + requirements schema.

Order (Safe View)

A minimal, non-PII order status view:

- status, totals, currency, tour date, and canonical references.
-

C.6 Economics & Discount Rules (OWN vs PARTNER + agent discount)

C MUST apply discount/commission deterministically based on:

- product_source per item: `OWN` | `PARTNER`,
- campaign context: `coupon` | `deeplink` | `travel_agent` | `agent_assistant` (if enabled),
- global settings from the Partnerships module (Point 7).

Agent/Bot discount (optional, but supported cleanly):

- Define a campaign type: `agent_assistant`
- Global setting: `agent_assistant_discount_percent` (default suggested: 5%)
- Applied only when request is authenticated (`agent_api_key`) and explicitly requests it (e.g., `apply_agent_discount=true`).
- Must be visible in Quote under `applied_rules` so the agent can “see” the cheaper final price *before checkout*.

Transparency requirement:

Quote MUST return `applied_rules` per item:

- `product_source`
 - `campaign_type`
 - `customer_discount_pct`
 - `partner_commission_pct` (if applicable)
 - `rule_source` (global settings version)
-

C.7 Currency & FX Rules (deterministic)

- Quote currency is determined by **site settings / user locale** (as defined in your platform).
- If cart is priced in a non-EUR currency, the system must compute:
 - commercial totals in checkout currency, and
 - settlement/reporting values in EUR **using the same FX source/rate rules as the website** (single SSOT).
- Quote MUST include:
 - **currency**,
 - **fx_rate_id** or **fx_timestamp** (traceability),
 - **totals** in checkout currency.

C.8 Requirements Schema (structured; UI↔contract parity)

Quote MUST return a structured, machine-readable **requirements[]** schema so agents collect exactly what is required.

Example:

```
"requirements": [

  {"id":"customer.first_name","label":"First
name","type":"string","required":true},

  {"id":"customer.last_name","label":"Last
name","type":"string","required":true},

  {"id":"customer.email","label":"Email","type":"email","required":true},

  {"id":"customer.phone","label":"Phone","type":"phone_e164","required":true},

  {"id":"traveler.dob","label":"Date of
birth","type":"date","required":true,"repeat":"per_traveler"}
```

]

Hard rules:

- Requirements **MUST** be declared at Quote time.
 - Checkout UI **MUST** render exactly the same required fields (**UI↔contract parity**).
 - For **OWN products**, the default should be minimal (first/last/email/phone), unless you choose to require more.
 - For **PARTNER products**, additional fields may be required by supplier; they must appear in Quote requirements.
-

C.9 Price Determinism + Revalidation Policy (fail-closed)

Quote **MUST** include:

- `pricing_hash` (or `pricing_version`) — signature of the priced snapshot
- `revalidate_policy: ok | sold_out | price_changed`

At checkout finalization, Rosotravel revalidates:

- If `sold_out` → abort and require new quote/date selection.
 - If `price_changed` → require explicit confirmation (new quote/session).
-

C.10 Core Endpoints (final names)

Create Quote

- `POST /api/agent/quotes`
Request (minimal):
- `items[]`: each includes `entity_id`, `date`, optional `time`, optional `variant_id`, `pax`, `locale`

- `campaign_context: { coupon_code? , deeplink_id? , travel_agent_user? , apply_agent_discount? }`
Response (must include):

- `quote_id`
- itemized totals + currency
- applied_rules per item
- requirements[]
- pricing_hash
- revalidate_policy
- expires_at (quote TTL)

Create Checkout Session

- `POST /api/agent/checkout-sessions`
Request:
- `quote_id`
- optional: `customer_prefill` (ONLY if allowed and minimal; never store extra PII unnecessarily)
Response:
- `session_id`
- `checkout_url` (signed, Rosotravel domain)
- requirements[] (echo; parity check)
- pricing_hash (echo)
- expires_at

Read Checkout Session

- `GET /api/agent/checkout-sessions/{id}`
Response:

- status: `created` | `opened` | `paid` | `expired` | `failed`
- if paid: `order_id` (safe)

Read Order (safe view, no PII)

- `GET /api/agent/orders/{id}`
Response:
- `order_status`: `paid` | `confirmed` | `cancelled` | `refunded` | `disputed`
- canonical references (`entity_id/canonical_url`), totals, currency, `tour_date`
- no customer PII

Get Cancellation Link (self-service only)

- `GET /api/agent/orders/{id}/cancellation-link`
Response:
- `cancellation_url` (signed, Rosotravel domain, customer action)
Hard rule:
- in v1, **no agent-driven cancel mutation endpoint.**

C.11 Abuse Protection, Logging, and Audit

Mandatory controls:

- rate limiting per key + IP
- anomaly detection (burst, pattern abuse, high fail ratio)
- allowlist / WAF rules for Mode B
- audit log includes:
 - key identity, scopes, endpoint, parameters size, response code
- **No PII in logs.**

C.12 Definition of Done (C)

C is complete when:

- An agent can deterministically: Quote → CheckoutSession → user pays via signed Rosotravel checkout URL.
- Quote exposes structured requirements and applied_rules.
- No supplier leakage is possible by contract and by allowlist of returned fields.
- Order status can be checked without exposing customer PII.
- Cancellation is customer self-service link only (v1).

Headless Checkout for Bots (Bots-Only Checkout UX + Contract)

Purpose

D defines the **checkout UX and behavior** optimized for bot-assisted purchases. Scope is **bot checkout only** (human UX is handled elsewhere).

Goals:

- minimal friction (fewest steps),
- wallet-first payment options,
- strict **UI↔API parity** with Quote requirements,
- supplier-safe (no Viator API exposure),
- deterministic outcomes and clear failure handling.

Entry Point

Checkout is initiated only via:

- `checkout_url` returned from `POST /api/agent/checkout-sessions`.

Hard rule:

- checkout_url MUST be signed, time-limited, and bound to session_id + pricing_hash.
-

Wallet-first behavior (ties to Point 6)

The bot checkout must prioritize:

- PayPal Express
- Apple Pay
- Google Pay

Rules:

- show only methods eligible for the user's device/browser (Apple Pay only where supported),
 - wallet options appear as primary CTAs above card payment (if card exists),
 - do not show payment methods not configured for the checkout currency/region.
-

Data Collection (minimal, supplier-driven)

Checkout must render exactly the fields from `requirements[ ]` returned by Quote (UI↔**contract parity**).

Default minimal set (OWN products):

- first_name, last_name, email, phone (E.164 with country code)

Partner/supplier expansion:

- additional traveler fields appear only if Quote requires them (DOB, passport, etc.).

Hard rules:

- No “surprise” required fields can appear in checkout.
- Phone must be validated as `phone_e164` (country code required).

UX Flow (bot-friendly)

1. **Confirm selection** (summary)

- items, date/time, pax, price totals, cancellation summary
- show applied discount (if any) transparently

2. **Provide required traveler/customer data**

- render requirements form
- inline validation

3. **Payment**

- wallet-first CTAs (PayPal/ApplePay/GooglePay)
- fallback card payment
- show price lock / revalidation note (if applicable)

4. **Confirmation**

- success page returns:
 - order reference
 - voucher link (if applicable)
 - “manage booking” link

D.6 Revalidation outcomes (must be user-clear)

If revalidation fails at finalization:

- **sold_out**: show “Sold out on this date/time” + link to retry quote/date selection.
 - **price_changed**: show updated price and require explicit acceptance before proceeding.
-

D.7 Post-purchase actions (v1 safety)

Bot/agent does not mutate bookings.

Provide **customer self-service links**:

- cancellation link (where policy allows)
 - manage booking link (customer authenticated view)
-

D.8 Security & Anti-abuse

- checkout_url is single-purpose, signed, time-limited
 - CSRF protection on forms
 - strict field allowlist from requirements schema
 - session state stored server-side only
 - no exposure of supplier endpoints in the client
-

D.9 Definition of Done (D)

D is complete when:

- checkout renders and validates requirements exactly as in Quote
- wallet-first payment works and is device-eligible
- sold_out/price_changed handled cleanly
- minimal data collection works for OWN products, and supplier-required expansion works for PARTNER products
- no agent-driven cancellation/reschedule endpoints exist in v1

a. Agent/LLM Discount (optional, recommended if used as a conversion lever)

Rosotravel MAY introduce an “Agent/LLM discount” to increase conversions initiated via AI agents (e.g., “Buy via agent and save 5%”).

Rule: discount must be visible pre-checkout (Quote stage).

The discount MUST be applied and shown in the **Quote** response, not only in checkout, so the agent can present the final price in the conversation and the user can make a decision based on price.

Eligibility (hard rules)

The Agent/LLM discount is applied only if ALL are true:

- request is authenticated (server-to-server) with a valid `agent_api_key`,
- `campaign_type = agent_assistant` (explicit),
- discount is enabled in global settings,
- `product_source` rules allow it:
 - **OWN products:** allowed by default,
 - **PARTNER products:** allowed by default

Discount computation

- Applied **per order item** based on `product_source` (OWN vs PARTNER).
- Discount MUST be reflected in:
 - item totals,
 - order totals,
 - and `applied_rules` (transparent breakdown).

Quote MUST return:

- `applied_rules` per item including:
 - `campaign_type = agent_assistant`
 - `customer_discount_pct`
 - `product_source`
 - `rule_source_version`

- `price_before_discount` and `price_after_discount` (or equivalent fields) for transparency.

Conflict resolution / stacking (hard)

Discount stacking must be deterministic:

1. Partner coupon (affiliate) wins and overrides agent discount (or disables it) — choose one policy and enforce globally.
2. Travel-agent login discount has priority over agent discount (trade rates must not stack accidentally).
3. If an item is PARTNER and discounting is not allowed → agent discount for that item is 0.

Abuse prevention (hard)

- Agent discount must NOT be accessible via URL params alone.
- It must require authenticated agent request (Mode B server-to-server).
- Use rate limits + anomaly detection on quote/session creation.
- Log applied discounts per session/order for audit.

Outcome: the agent can truthfully say “via this assistant you get X% off” and the user sees the same price in checkout (UI↔contract parity).

V. Delivery, Migration & Non-Functional Requirements

Important clarification: all of those features must be a big patch of the current website built on Laravel www.rosotravel.com. You don't build a website from scratch.

Only the front-end should be built from zero on totally new technologies.

5.1 One-Time Migration & Initialization Processes

Changes for current website

 Website Rosotravel MVP - featurer list 2025 y. here you can see screenshots and a feature list of the current website.

You can test and see the backend of how to add products on www.rosotravel.at which is our test environment and this website is not live. (not www.rosotravel.com which is a live website).

- Add new: Last tour snapshot: here can be edited last tour snapshot and besides automatically showing reviews from latest tours you can add here admin photo or video.
- Adding reviews: customers while adding reviews can add video or photo from a tour
- Other panel change required based on this features

a) New UI for adding products and migration

Internal Products → Model C Parity Migration, new UI is described in chapter 3 - [3.3X Internal Products](#)

- migration_state lifecycle (NOT_STARTED / IN_PROGRESS / COMPLETED / BLOCKED)
- migration queue and prioritization (per city, tier, revenue)
- auto-prefill rules from existing internal product fields
- manual completion rules (what must be filled in “Model C Classification”)
- Set blocking cutover rules (no Set eligibility until COMPLETE)
- reporting dashboards (coverage, blocked count, reasons)
- rollback rules and audit requirements

b) Last Tour Snapshot

Concept Overview

Rosotravel currently has live products, attractions, cities, countries and blogs published on www.rosotravel.com.

These existing entities must be migrated into the new AI & SEO programmatic system without creating any “legacy mode” or special handling logic.

Launching a new website starts from a MIGRATION of tours, blog, and other pages.

The migration is handled as a one-time Initial Content Re-Ingestion process, where all existing content is treated as RAW input data, identical in structure and lifecycle to newly ingested content from Viator or future internal products.

Isolated Test Environment

The entire current version of the website is first copied to an isolated test domain:

www.rosotravel.at

This domain is used exclusively for:

- schema mapping
- RAW batch creation
- AI / SEO pipeline testing
- prompt and QA iteration

No production publishing or SEO risk exists at this stage.

RAW Batch Creation (Core Principle)

All existing entities from the current site are:

- extracted
- mapped to the new unified RAW schema
- stored as RAW batches, grouped by city

Examples:

- RAW_BATCH_VIENNA_v1
- RAW_BATCH_PARIS_v1

Each city RAW batch may contain:

- existing internal products
- newly added internal products
- products later ingested from Viator

All products inside a batch are treated equally by the system.

Testing Strategy

For validation and iteration:

- only one selected RAW batch (e.g. Vienna) is fully processed
- a complete AI + SEO run is executed for that batch only
- all other RAW batches remain unprocessed and unpublished

This allows:

- controlled testing
 - fast iteration
 - zero impact on production SEO
-

Countries, Regions, and Higher-Level Destinations

Countries, regions, islands, and similar entities:

- are not part of product RAW batches
- are handled separately, in line with the existing scope
- are loaded independently (as authoritative entities)

This is not a limitation, as the current site does not contain regions or islands.

Final Outcome

After validation:

- the same RAW data
- the same AI versions
- the same keyword ownership
- the same URL structures

can be published on www.rosotravel.com as fully processed, production-ready content.

No special migration logic remains in the system.

Case 1: Final Summary of migration

Legacy content migration is treated as a one-time, non-recurring process.

All existing content currently published on www.rosotravel.com — including:

- products (tours),
- attractions,
- cities,
- and countries,

is copied to the staging environment www.rosotravel.at and ingested into the new system as RAW entities, exactly as if they were newly imported from Viator or created natively.

After migration:

- legacy products, attractions, cities, and countries use the same data schema, fields, and versioning model as all future content,
- they enter the same RAW batch structure (e.g. `vienna_raw_batch_v1`),
- they go through the identical AI, SEO, enrichment, and publishing pipelines.

From this point onward, the system does not distinguish between migrated content and newly ingested content.

The exact technical method of mapping and importing legacy data (scripts, ETL, API-based import, manual tooling) is intentionally out of scope and left to the implementation team, as long as the final state conforms to the unified RAW ingestion model defined in this specification.

Case 2: SEO MIGRATION (LEGACY URL HANDLING)

This section defines the one-time SEO migration rules and clearly separates engineering responsibilities from SEO team responsibilities. The goal is to migrate the entire existing Rosotravel website to the new architecture without any legacy URLs remaining indexable.

2.1 Migration Principles (Locked)

These principles are final and apply to the entire migration:

- All legacy URLs must be preserved and handled
 - No selective pruning.
 - Every existing indexable URL must have an explicit outcome.
- Legacy blog posts migrate 1:1
 - No blog retirement during migration.
 - No merging or consolidation at this stage.
- Legacy city pages map to new city pages
 - City pages do not redirect to things-to-do hubs
 - Mapping city to city
- No legacy URL remains indexable
 - All legacy URLs must be redirected or retired.
 - No legacy page may return 200 or be present in sitemaps.

2.2 Responsibilities — Engineering (Appinventiv)

The following items are in scope for engineering.

2.2.1 Redirect Infrastructure

Engineering must provide:

- A scalable redirect system supporting:
 - one-to-one redirects,
 - many-to-one redirects.
- Redirect type:
 - 301 Permanent Redirect only.
- Constraints:
 - no redirect chains,
 - max one hop per URL,
 - redirect target must equal canonical URL.

2.2.2 Canonical & Indexation Enforcement

Engineering must enforce:

- Canonical tags always point to:
 - new URL only.
- Legacy URLs:
 - never appear in XML sitemaps,
 - never appear in AI sitemap or AI feeds,
 - default to **noindex, follow** until redirected.
- No legacy page may:
 - be indexable,
 - be canonicalized to itself.

2.2.3 Sitemap & AI Feed Regeneration

Engineering must ensure:

- On publish / unpublish:
 - standard sitemaps update correctly,
 - AI sitemap updates,
 - AI summary feed updates.
- Only new canonical URLs are eligible.
- Legacy URLs are fully excluded.

2.2.4 Validation & Safety Defaults

Engineering must implement:

- Safe default:
 - any unmapped legacy URL → **noindex, follow**.
- Logging:
 - unmapped legacy URL hits,
 - redirect errors,
 - canonical mismatches.

2.3 Responsibilities — SEO Team (Out of Scope for Engineering)

The following activities are explicitly handled by the SEO team and are not part of development scope.

2.3.1 Legacy URL Inventory

SEO team must:

- Export full list of:
 - legacy product URLs,
 - legacy attraction URLs,
 - legacy city / country URLs,
 - legacy blog URLs.
- Validate indexation status before migration.

2.3.2 URL Mapping Decisions

SEO team must:

- Decide mapping for each legacy URL:
 - old → new (1:1),
 - many → one (consolidation).
- Provide final redirect mapping file:
 - legacy URL → target canonical URL.

Engineering only implements, never decides mapping logic.

2.4 Explicit Exclusions

The following are out of scope for the platform build:

- Manual URL pruning decisions.
- SEO content consolidation strategy.
- Post-migration ranking analysis.
- Backlink outreach or remediation.

2.6 Migration Completion Criteria

Migration is considered complete when:

- 100% of legacy URLs return:
 - 301 redirect, or
 - 410 (pre-approved only).
- No legacy URL is indexable.

- New sitemaps contain only new URLs.
- AI sitemap and AI summary feeds contain only new entities.

- SEMrush governance → tylko 3.6
- Indexation + Viator compliance + sitemaps eligibility → tylko 3.5
- RAW tables + versioning + hashes + diff → tylko 4.3–4.4
- Embeddings / Topic graph / Anti-cannibalization → tylko 4.5
- AI drafting → tylko 4.6
- Validation → tylko 4.7
- Translation → tylko 4.8
- Schema/tokens → tylko 4.9
- LLM feeds / AI sitemap / /ai-query → tylko 4.10
- MCP → tylko 4.11
- Workflow top-level → tylko 4.1 (krótkie, bez reguł – same kroki + odwołania)

W innych miejscach zostawiasz wyłącznie 1–2 linijki “See: X.Y”.

5.2 Testing environment on subdomain (rosotravel.at)

Concept

In short implement scopes 1, 2 and 3 into domain www.rosotravel.at, make a proper test, if everything works properly. Migrate whole content and solution to www.rosotravel.com.

Rosotravel.at we should make test ONLY on 1 city bath which will be published and tested properly, blog, guide site, everything tested and then migrated

- We can fulfill [rosotravel.at](https://www.rosotravel.at) with some content including One-Time Migration & Initialization Processes

Rosotravel will use www.rosotravel.at as a **dedicated staging and experimentation environment** for the entire platform.

All new functionality, content generation, AI pipelines, SEO automation, and data ingestion will be:

- developed,
- tested,
- iterated,
- and validated

exclusively on rosotravel.at first.

Only after the system is stable, content is complete, and QA is finished, the same data, versions, and configurations are deployed to the production domain www.rosotravel.com.

This applies to:

- programmatic SEO pages
 - AI-generated content
 - images
 - schema
 - feeds (AI sitemap, AI summary, explore dataset)
 - new product ingestion (internal + Viator)
-

Operational Principle

- rosotravel.at is treated as a **full mirror of production**, not a partial sandbox.
- The same CMS, pipelines, prompts, and rules apply.
- The only differences are:
 - domain name
 - environment configuration (staging vs production)
 - publishing targets

This allows long-running tests (weeks/months) without any SEO or revenue risk.

Deployment Flow

1. All development and content generation runs on **rosotravel.at**
2. Selected batches and features are validated end-to-end
3. Once approved, the same:
 - RAW data
 - AI versions

- keyword ownership
 - URL structures
 - schema
4. Are deployed to **rosotravel.com**, where the platform continues operating normally
-

API & External Services Consideration (Open Question)

The scope intentionally flags the following for technical validation:

- Do any external services (e.g. **SEMrush API**, GSC, GA4, Viator, image/CDN providers) enforce **domain-level restrictions**?
- If so, can this be handled by:
 - separate API keys per environment, or
 - environment-based configuration (staging vs production)?

Initial assumption:

- API keys may need to be duplicated or reconfigured per domain
- No fundamental blockers are expected, but this must be confirmed during technical setup

This question is **explicitly left for the implementation team** to validate and document before launch.

Scope Boundary

This staging strategy is:

- a **platform-level operating principle**
- **not part of the SEO execution pipeline**
- **not a recurring workflow**

It exists to reduce risk, enable experimentation, and allow slow, high-quality iteration before production exposure.

5.3 Security, Access Control & Roles

04. API-Level Security

Every API endpoint must include:

1. Role-based access (RBAC)

Example:

- `/ai/regenerate` → Admin + SEO Lead
- `/schema/rebuild` → Admin + Developer
- `/keyword/assign` → SEO Lead only
- `/nearpage/approve` → Admin + SEO Lead

2. Signed requests

Endpoints that trigger AI or write to database must require:

- HMAC signature
- 15-second expiry
- Nonce to prevent replay

3. Audit logs for every API call:

- user_id
- endpoint
- before/after diff
- timestamp
- success/failure

5.4 DevOps Monitoring (Internal Only)

Used only by Appinventiv/DevOps, not visible to editors:

Monitors:

- Qdrant latency / uptime
- External API errors (GSC, SEMrush, GPT, Google Places, Wikidata)
- Queue lengths

- Failed jobs
- Schema validation errors

DevOps alerts only:

- No dedicated panel for manual interaction
- Logs forwarded to Slack / email

5.5 Publication limits / GOOGLE DOWNGRADE + backpressure

- Staged publishing (5–10k pages, then 48h pause). Vey important limit to describe exactly
- Real data on every page (Duration, Languages, Meeting Point, Availability).
- No supplier copy; AI paraphrases from structured facts.

5.6 NFR (SLO/SLA/cache/failure modes

This section defines **mandatory, non-functional requirements** for the Rosotravel platform.

These requirements are **not optional** and apply across all layers: SEO, AI Drafting, Dynamic Experience Engine (DEE), LLM feeds, MCP Server, and booking flows.

Their purpose is to ensure:

- scalability under organic traffic and LLM crawler traffic,
- operational stability and fault tolerance,
- SEO and indexation safety,
- long-term maintainability of AI-driven systems.

Core Technology Stack

Database layer

The primary database must be PostgreSQL (preferred) or MySQL.
It stores:

- products, cities, attractions (POI),
- users (where applicable),
- publishing state,
- keyword ownership,
- versioning metadata.

Hard rule:

The primary database must never be a single point of failure for organic traffic, crawler traffic, or LLM access.

Caching layer (mandatory)

Caching must be applied aggressively and systematically across the platform.

Redis

Redis is mandatory and must be used for:

- session storage,
- caching database query results,
- caching JSON fragments (product payloads, listings, AI feed objects).

CDN (Cloudflare, CloudFront, or equivalent)

The CDN must cache:

- static pages,
- listing pages (things to do, attractions),
- AI feeds:
 - `/ai-sitemap.xml`
 - `/ai-summary.json`

- `/api/tours/feed`

Hard rule:

The platform must remain responsive even if the database is under heavy load or temporarily unavailable.

Scalability & application architecture**Stateless application design**

The Laravel application must be stateless:

- no session data stored on application instances,
- sessions stored in Redis or database,
- instances must be horizontally scalable behind a load balancer.

API-first architecture

- All frontend interactions must be served via APIs.
- Frontend must be replaceable without rewriting business logic.
- No tight coupling between frontend rendering and backend logic.

Job queues & background processing

Queues must support:

- minimum 3 workers running in parallel,
- independent scaling of workers.

Designed throughput (baseline targets):

- $\geq 10,000$ AI jobs per day (generation, rewrites, QA regeneration),
- $\geq 100,000$ feed diff events per day (price, availability, metadata changes).

Availability & SLA

Uptime targets

- Initial phase: minimum 99.5% monthly uptime,
- Post-stabilization target: 99.9% monthly uptime.

This applies to:

- frontend,
- APIs,
- LLM feeds,
- MCP Server (read-only).

Maintenance windows

- Planned maintenance only during low-traffic hours.
- Frontend must display clear maintenance messaging.
- Maintenance must never silently return errors to crawlers.

External dependency failures

If external providers fail (e.g. OpenAI, Viator):

- jobs must not crash the system,
- jobs must be routed to a Dead Letter Queue (DLQ),
- retries must be possible after recovery.

Reliability & error handling

Retry & Dead Letter Queue (DLQ)

All critical jobs must:

- have retry logic (recommended 3–5 attempts),
- end in DLQ after retry exhaustion,

- log failure reason and payload metadata.

Critical jobs include:

- feed imports,
- AI content generation,
- sitemap and AI feed builds,
- publishing operations.

Feed Diff Engine (mandatory)

The Feed Diff Engine must:

- detect field-level changes (price, availability, metadata),
- not trigger full product regeneration if only price or availability changes,
- log:
 - which fields changed,
 - which downstream modules were notified (AI, SEO, booking).

Atomic publishing (consistency guarantee)

Publishing must be atomic:

- a page is either fully published or not published at all,
- no half-published or inconsistent states are allowed,
- on failure, the previous version remains live.

Security & compliance

Transport security

- All traffic over HTTPS (TLS 1.2+),
- HSTS enabled.

Secrets & access

- API keys and secrets stored in a secure secret manager (Vault / Secret Manager),
- no secrets hard-coded in repositories.

Personal data & GDPR

- No storage of payment card data (use PSP such as Stripe or Adyen),
- GDPR compliance required:
 - right to account deletion,
 - transparent privacy policy.

Rate limiting

Mandatory rate limiting on:

- public APIs,
- feed endpoints,
- `ai-query`,
- MCP Server tools.

Limits must be configurable per IP or key.

SEO technical hygiene (NFR)

Important clarification:

This section defines technical SEO invariants and service-level guarantees only.

All indexation decisions, canonical routing logic, intent ownership, and page-level SEO governance rules are defined separately in the “SEO Governance & Indexation Control” section and are enforced by the publishing layer.

Sitemaps

- Full sitemap regeneration maximum every 24 hours.
- `lastmod` updated within 1 hour of publishing or unpublishing any indexable page.

Robots / Canonical / Hreflang (technical guarantees)

- Every indexable page must contain a canonical tag.
- Canonical URLs must be clean, absolute, and parameter-free.
- Correct `hreflang` must exist for all language variants of the same page.
- Canonical must always point to the same language version (never cross-language).

Dynamic parameters & UI variants

- All filter parameters and UI-only URL variants must be:
 - `noindex, follow`,
 - canonicalized to the clean base URL.
- No exceptions are allowed.

Schema.org technical validity

- Required schema types must be present on all applicable pages.
- 100% of pages of each type must pass Google Rich Results Test without critical errors.
- Schema generation failures must block publishing.

Observability & monitoring

Logging

- access logs,
- error logs,
- job execution logs (AI, feed, SEO).

Metrics

At minimum:

- response time (P50, P90, P95),
- job queue size,
- job processing rate,
- error rate per hour.

Alerting

Automatic alerts when:

- error rate > 2% for 10 minutes,
- job queue backlog exceeds threshold for more than 30 minutes,
- sitemap or AI feed generation fails.

Dashboards

A single operational dashboard (Grafana, Datadog, or NewRelic) must show:

- request latency,
- error rate,
- job throughput,
- feed generation status.

Data quality & versioning

Versioning

- Every product entity must have a version number (v1, v2, v3...).
- Feed Diff Engine always compares current versus previous version.

AI content history

- Minimum last 3 versions of AI-generated content must be stored.
- Enables audit, rollback, and quality comparison.

QUALITY SYSTEM (AI + MANUAL REVIEW)

- Minimum 3% of newly published English pages must enter a QA queue.
- The system must allow:
 - marking content as OK / Fix / Regenerate,
 - linking feedback to prompt version and model version.

1) AI Scoring

- **Readability, semantic uniqueness (distance between pages), factuality vs feed.**

2) Manual Sampling

- **For every ~1,000 generated pages review 10–20; log feedback; model learns rules.**

3) Regeneration of Low Quality

- **Auto re-gen if CTR < 1% or Bounce > 70% on target templates.**

Strategic outcome

This NFR layer ensures that:

- AI and SEO scale safely,
- LLM crawlers and agents do not destabilize the system,
- content quality is auditable and reversible,
- Rosotravel can grow programmatically without accumulating hidden technical or SEO debt.

5.7 Partnerships & public docs

Purpose

Mutual feeds with museums/attractions/operators; schema **Organization** with **sameAs**.

Backend

- Feed mappers; sync jobs.

Front-End

- Partner attributions.

DoD

- At least one pilot partner feed live.

VI. New Payment Processor

Payments for Agent-Friendly Checkout (Wallets)

Rosotravel must support wallet-first payments that minimize friction for agent-driven purchases **However, those methods will be available also for human customers buying on the website :**

- PayPal Express Checkout
- Apple Pay
- Google Pay

Requirements

- These payment methods must be available in the **agent checkout flow** (**/checkout/agent?cs=...**) and exposed as the fastest payment options where device/browser eligibility allows.
- Payment confirmation must trigger the same **final availability revalidation + order creation** as standard checkout.
- No changes to public feeds (B) are required; this is a checkout/payment layer only.
- Rosotravel will provide the IT vendor with access to the required payment accounts; the vendor configures gateways/providers accordingly.

PayPal Express, Apple Pay, Google Pay

These must be configured in the PSP/payment stack so they can be used as primary methods in **bot checkout (D)** and optionally also in standard checkout. Rosotravel provides the IT vendor access to required merchant accounts/configuration; vendor implements and validates eligibility rules per device/browser and currency/region.

VII. Rosotravel Partner Programs v1 (Affiliate & Travel Agent System)

VII. Rosotravel Partner Programs v1 (Affiliate & Travel Agent System)

7.1 Purpose and Scope

This section defines the complete functional scope for Rosotravel Partner Programs implemented as one unified Partners module, covering:

A) Affiliate / Creators Program (commission model)

Partners promote Rosotravel using a coupon code and/or tracked deeplinks. Commissions are earned after the tour is completed (travel date) and paid monthly against invoices.

B) Travel Agents Program (trade discount model)

Travel agents receive an automatic discount when logged in. No commission ledger, no invoices, no payouts.

C) Gooru Guide Program (content + revenue share model)

Selected guides contribute city-scoped informational/blog content under Rosotravel SEO governance and earn revenue share from attributable purchases driven by their content, subject to freshness rules.

The system must support different economics for OWN products vs PARTNER-sourced products, controlled via globally configurable settings in a dedicated Partners Admin UI.

Hard boundary: This module must not leak supplier APIs or restricted supplier payloads. Partner attribution and settlement must remain Rosotravel-owned and auditable.

7.2 Key Definitions

- OWN product: Operated by Rosotravel (e.g., Rosotravel Originals).
- PARTNER product: Sourced from external suppliers/APIs; Rosotravel resells/aggregates under Rosotravel canonical pages.
- Program type: One of the allowed partner program types: `affiliate_commission`, `travel_agent_discount`, `gooru_guide_program`.
- Campaign type: Attribution channel used for economics and conflict resolution:
 - `affiliate_coupon`
 - `affiliate_deeplink`
 - `gooru_content`
- Coupon campaign: Partner code applied at checkout; may grant customer discount and sets attribution.
- Deeplink campaign: A tracked link identified by `deeplink_id` (and `partner_id`) that can attribute purchases without requiring a coupon code (cookie window). Each deeplink MAY point to a specific Rosotravel canonical target URL (city, attraction, things-to-do, product, blog article, bundle). Deeplink targets are stored server-side and validated to prevent open-redirect abuse.
- Content attribution (Gooru): A first-party attribution cookie set on article view that can attribute purchases to a Gooru Guide if eligibility rules are satisfied (freshness).
- Pending: Commission exists, but is not yet eligible (before earning delay passes).

- **Earned:** Commission becomes eligible after `travel_date + earned_delay_days` (default 7).
- **Statement month (monthly close):** A locked monthly period that shows the exact amount the partner must invoice for that month.
- **Settlement currency:** EUR (statements/payout thresholds are evaluated in EUR). Sales may occur in other currencies (see FX rule).
- **Topic Pool:** A city-scoped queue of pre-approved informational topics derived from SEMrush keyword retrieval outputs written to `keyword_map`.
- **Topic claim:** A reservation of a Topic Pool item by a single guide that triggers keyword ownership locking (audited) and prevents duplicates.

7.3 Program Economics Matrix (Default Global Settings)

All discounts and commissions are computed per order item based on `product_source`. Mixed carts (OWN + PARTNER) must apply rules item-by-item.

Program / Campaign / Source → Customer Discount → Partner Commission
→ Commission Base

Affiliate / Coupon / OWN → 10% → 20% → `item_amount_after_discount`

Affiliate / Deeplink / OWN → 0% → 25% → `item_amount`

Affiliate / Coupon / PARTNER → 5% → 5% → `item_amount_after_discount`

Affiliate / Deeplink / PARTNER → 0% → 10% → `item_amount`

Gooru Guide / Content / OWN → 0% → 5% → `item_amount`

Gooru Guide / Content / PARTNER → 0% → 5% → `item_amount`

Travel Agent / Login discount / OWN → 20% → 25% → 30%

Travel Agent / Login discount / PARTNER → 10% → 11.5% → 13%

Commission base definition (hard):

- item_amount means the charged item amount used for settlement calculations.
- Default v1 rule: item_amount excludes VAT/taxes if taxes are itemized; otherwise use the charged amount.
- Payment processing fees are not deducted in v1 unless explicitly configured later.

7.4 Global Settings (Editable in Partners Admin UI)

Admin must be able to change discount/commission values globally from the Partners module. These settings must NOT be managed in the existing Bookings → Discount Codes UI.

7.4.1 Global Settings Groups

Affiliate — OWN

- coupon_customer_discount_percent (default 10)
- coupon_partner_commission_percent (default 20)
- deeplink_partner_commission_percent (default 25)
- deeplink_customer_discount_percent (default 0)

Affiliate — PARTNER

- coupon_customer_discount_percent (default 5)
- coupon_partner_commission_percent (default 5)

- deeplink_partner_commission_percent (default 10)
- deeplink_customer_discount_percent (default 0)

Gooru Guide — Content (OWN)

- gooru_content_partner_commission_percent_own (default 5)
- gooru_content_customer_discount_percent_own (default 0)

Gooru Guide — Content (PARTNER)

- gooru_content_partner_commission_percent_partner (default 5)
- gooru_content_customer_discount_percent_partner (default 0)

Travel Agent — Discounts

- travel_agent_discount_percent_own (default 25)
- travel_agent_discount_percent_partner (default 10)

Settlement & Timing

- earned_delay_days (default 7)
- minimum_payout_threshold_eur (default 200)
- invoice_deadline_day_of_month (default 3)
- payout_day_of_month (default 10)
- monthly_close_day_of_month (default 7) (hard: closes prior month + resets counters)
- deeplink_cookie_window_days (default 30)
- gooru_content_cookie_window_days (default 30)

Gooru Guide — Topic & Freshness Controls

- `topic_claim_ttl_days` (default 14)
- `max_active_claims_per_guide` (default 2)
- `gooru_freshness_required_days` (default 120)
- `gooru_auto_publish_noindex_until_seo_approved` (default true)

7.4.2 Override Hierarchy (v1)

v1 supports global settings only. Partner-specific overrides may be added in v2.

Computation order: `product_source` → `program_type` → `campaign_type` → global settings.

7.4.3 Explicit Separation from B2C Discount Codes (hard)

Partner coupon codes used for attribution and commission must be managed ONLY under Partners → Affiliate Program.

Existing Bookings → Discount Codes remain B2C promotions and do not create attribution or commissions.

7.5 Program Types and Onboarding

7.5.1 System program types (hard)

System supports only these program types:

1. `affiliate_commission` (subtype: creator/influencer/blogger/guide/b2b)
→ same marketing form aliases: `/b2b/affiliate` and `/b2b/creators`
2. `travel_agent_discount`

→ separate form: /b2b/travel-agents

3. gooru_guide_program

→ profile can be created only by admin (no self-serve in v1)

7.5.2 Onboarding Paths

Path A — Self-serve application (apply → in_review → approved):

- Applicant submits form with website/social links and company details.
- Admin reviews and approves/rejects.
- Upon approval: partner user receives invite email to set password.
- Affiliate: coupon + primary deeplink auto-generated and shown in portal.
- Travel agent: login discount enabled; no coupon/deeplinks.

Path B — Admin-created account (manual creation):

- Admin creates affiliate / travel agent / gooru guide in backoffice and selects program type.
- System sends invite email to set password (no passwords by email).
- Affiliate: auto-generate coupon + primary deeplink.
- Travel agent: no codes; login discount only.
- Gooru guide: auto-generate primary deeplink (and enable Gooru portal access).

7.6 Affiliate Program Functional Requirements

7.6.1 Partner assets: Coupon Code + Deeplinks (always available)

Affiliate partners must always have:

A) One coupon code

- Auto-generated on approval; globally unique within the Partners module.
- Admin may edit the code (audited). Partner can only request changes.
- Coupon code format (default rule):
 - {AAA}{NN}{AFI}
 - AAA = first 3 letters of partner first name (A–Z only, uppercase, diacritics stripped; if fewer than 3 letters, pad with “X”)
 - NN = incremental numeric suffix that guarantees uniqueness (e.g., 10, 11, 12...)
 - AFI = fixed program tag for affiliate program
- Hard rule: the system **MUST** never generate a duplicate coupon code. Uniqueness is checked at creation time; if collision occurs, NN increments until unique.

B) Deeplinks (primary + campaign deeplinks)

- One primary deeplink is auto-generated on approval and shown in the portal (copyable URL; cookie attribution).
- In addition, the portal **MUST** provide a “Deeplink Builder” so the partner can generate multiple campaign deeplinks that point to a specific canonical Rosotravel page.
 - Partner selects a target using entity pickers (recommended): entity_id / canonical_url resolved server-side.
 - Allowlist enforced (hard): city / attraction / things-to-do / product / blog article / bundle.

- System generates a deeplink_id and stores target_url server-side.
- Deeplink resolves via a Rosotravel tracking route that:
 - 1) sets attribution cookie (deeplink_id + partner_id) for deeplink_cookie_window_days,
 - 2) redirects to the stored target_url (optionally appending allowed UTMs and non-binding hints).
- Allowed parameters (append-only, non-binding):
 - utm_source, utm_medium, utm_campaign, utm_content
 - Optional UI prefill hints: date=YYYY-MM-DD, pax=2, lang=en-US (hints must never bypass availability validation)
- Hard rule: deeplink targets MUST be Rosotravel canonical URLs only. No external redirects. No arbitrary URL inputs without server-side allowlist validation.
- Portal must show how code and deeplink work, including economics for OWN vs PARTNER products.

7.6.2 Attribution rules (hard)

At checkout/order creation, attribution resolves deterministically as:

1. If partner coupon used → coupon wins and overrides any deeplink/content attribution.
2. Else if an affiliate deeplink cookie exists and is valid → attribute to that affiliate partner (affiliate_deeplink).
3. Else if a Gooru Guide content attribution cookie exists and is eligible (freshness rule satisfied) → attribute to that Gooru Guide partner (gooru_content).
4. Else → no attribution.

Hard rules:

- Attribution MUST be computed per order item and remain auditable.
- Travel-agent login discounts do not create partner attribution (trade model only).
- If multiple cookies exist, the priority order above is enforced globally.

7.6.3 Commission ledger and earning (hard: after travel date, not booking)

On successful payment, for each order item with partner attribution:

- Determine product_source (OWN/PARTNER).
- Determine campaign_type (affiliate_coupon / affiliate_deeplink / gooru_content).
- Load global settings for product_source + campaign_type.
- Compute commission_base and commission_amount.
- Create ledger entry with status pending.
- Set earned_at = travel_date + earned_delay_days (default 7).
- Daily job converts pending → earned when earned_at passed and no refund/chargeback.

Hard clarification:

Commission eligibility is based on tour completion (travel_date) + delay. It is not earned at booking time.

Gooru Guide-specific ledger fields (additional):

- If campaign_type = gooru_content, ledger entry MUST store:
 - guide_partner_id
 - article_id (source page)

- gooru_freshness_eligibility_snapshot (eligible/ineligible at conversion time)

7.6.4 Refunds and chargebacks (hard)

If an order/item is refunded or chargebacked:

- Reverse pending/earned ledger entries.
- If already paid: create negative adjustment for next payout cycle.

7.6.5 Monthly close, statements, invoices, and “counter reset” (hard)

The partner must always see a closed monthly amount that tells them exactly what invoice to issue for a specific month, and then the next month starts from zero.

Monthly close behavior (hard):

- On monthly_close_day_of_month = 7, the system performs a monthly close for the previous calendar month:
 - locks a Statement for month YYYY-MM (previous month),
 - calculates totals from ledger entries with status = earned whose earned_at falls within that statement month,
 - applies all reversals/adjustments known up to close time,
 - produces a final invoice_amount_eur for that month.
- After the close completes, the portal:
 - shows “Closed month: YYYY-MM → Invoice amount: X EUR” (locked),
 - and resets the “Current month counters” to 0 for the new month cycle display.

What partner sees in the portal (Affiliate and Gooru):

- “Current month (running)” counters: Pending / Earned / Forecast (starts from 0 after close).
- “Last closed month” block: Month label + exact invoice amount + statement download.
- Statement history list: month-by-month (each locked on the 7th).

Invoice and payout timing (as per settings):

- Invoice must be uploaded by invoice_deadline_day_of_month (default 3).
- Payout is processed manually by Finance by payout_day_of_month (default 10).
- If earned_total_eur for the closed month < minimum_payout_threshold_eur:
 - the statement is still generated and visible,
 - but it is marked “carried forward” and not payable until threshold reached (carry-forward logic).

7.6.6 Data visibility (hard: no tour name, no customer PII)

Partner portal transaction list MUST show:

- order date, travel_date, destination city/country, offer_ref (offer_id/internal ref), base amount, commission, status.

MUST NOT show:

- customer name/email/phone, full product title.

7.6.7 Gooru Guide Program (Content + Revenue Share)

Purpose

Gooru Guide Program enables selected local guides to contribute destination content (blog/informational articles) under Rosotravel governance and earn revenue share from attributable purchases driven by their content.

Eligibility and account creation (hard)

- A Gooru Guide profile can be created only by admin (no self-serve applications in v1).
- Admin assigns:
 - program_type = gooru_guide_program,
 - one or more cities (scope),
 - writing language(s),
 - optional “guide tier” (used for moderation priority).
- System sends invite email to set password (never email passwords).
- System auto-generates a primary deeplink for the guide (as per deeplink rules). Additional campaign deeplinks may be created via Deeplink Builder.

Topic Pool source (auto from SEMrush keyword retrieval outputs)

- Topics for Gooru Guides MUST be derived from informational keywords discovered during SEMrush keyword retrieval and written to keyword_map (SSOT).
- A Topic Pool Generator job MUST run after each keyword retrieval/batch activation and create city-scoped topics using keyword_map records that match:
 - intent_primary = informational (or Mixed with informational as allowed by SEO rules),
 - owner_page_type = Blog Page,
 - status in (candidate / reserved) AND lock_state = unlocked,

- not already present as a topic and not already owned by another canonical URL.

- Each Topic Pool item stores:

- topic_id, city_id, primary_keyword, intent_type=informational, target_template=blog/informational, status.

- Hard rule: Topic Pool must not introduce new keywords outside keyword_map and must not bypass ownership rules.

Topic Pool and keyword locking (hard)

- Status lifecycle (minimum):

- available → claimed → draft → submitted → published_noindex → approved_indexed (or returned_for_fixes)

- Claiming a topic:

- When a guide clicks “Claim”, the topic becomes claimed and is assigned to that guide.

- The system MUST write a deterministic reservation to keyword_map (audit required) and lock the keyword to prevent duplicates (locking is mandatory).

- Once locked/claimed, the topic is hidden from other guides.

- Anti-stall controls:

- claim_ttl_days (default 14): if no submission within TTL, the system auto-releases the topic back to available and unlocks the reservation (audited).

- max_active_claims_per_guide (default 2): prevents hoarding.

Gooru Guide Portal (permissions)

A Gooru Guide user must have a dedicated portal experience that allows:

- Edit public bio/profile (name, short intro, expertise, languages, city assignment, social links).

- View an “Available Topics” list for assigned city/cities only.
- Claim a topic (shows TTL + rules) and create a draft.
- Write / edit / submit an article.
- Use Rosotravel Media Library (approved assets) and insert images via Media Picker.
- See content performance summary and commission status.
- Receive freshness reminders and see freshness eligibility status per article.

Authoring requirements (hard)

- Articles must be created using the approved Rosotravel article template rules (H1/H2/H3, intro, featured snippet block, FAQ/PAA block where applicable, internal link placement rules, and final on-page validation gates).
- Guide must insert internal links to Rosotravel tours/products using the portal’s internal-link picker (no raw external links for monetization).
- The system stores a full version history of edits (at least last 3 versions) for audit and rollback.

Publishing + SEO review (scalable safety)

- On submission, the system auto-publishes the article immediately, but with:
 - `indexation_state` = `noindex` (default) until SEO review approval,
 - canonical URL set,
 - internal links active,
 - QA flag: “pending SEO review”.
- SEO team receives the article in a review queue and can mark:
 - OK → switch to index,
 - Fix → return to guide with notes,

- Regenerate/Rewrite (optional) → creates a new draft version.
- Hard rule: the system must be able to batch-review and approve at scale (queue + filters by city/language/guide).

Revenue share (commission) model

- Commission rate (default): 5% of order item value for attributable purchases.
- Attribution mechanism:
 - When a user views a Gooru Guide article, the system may set a first-party content attribution cookie (gooru_content) containing guide_partner_id and article_id.
 - Attribution is applied only if no higher-priority partner coupon/deeplink exists (see 7.6.2).
- Freshness eligibility rule (hard):
 - A Gooru Guide article earns commission only if it has been updated by the guide at least once within the last 120 days.
 - If last_guide_update_at > 120 days ago, the article becomes ineligible until updated; new conversions from that article do not create commission entries during ineligible periods.
- Commission ledger:
 - Uses the same earning logic as affiliate (earned after travel_date + earned_delay_days).
 - Ledger entries must store: guide_partner_id, article_id, attribution_source = gooru_content.

Images and licensing (hard; scale-safe)

To prevent copyright/licensing violations at scale:

- MVP rule (hard): Gooru Guides may only insert images from Rosotravel Media Library (approved assets). No external image hotlinks.
- The Media Library must be accessible from the Gooru portal via a “Media” tab and also via inline “Insert image” picker in the editor.
- Hard rule: the system must store asset provenance fields for all media items (who uploaded/approved, license type, consent flag) for audit.

7.7 Travel Agents Program Functional Requirements

Travel agents receive automatic trade discounts when logged in. No commissions, statements, invoices, or payouts.

7.7.1 Approval and login discount (hard)

- Same apply/approve flow or admin-created account.
- After approval, travel agent logs in and discount is applied automatically in checkout per item based on product_source.
- Travel agents do not generate coupon codes or deeplinks.

7.7.2 Savings dashboard

Travel agent portal must show:

- Current discount rates for OWN and PARTNER products (from global settings).
- Total savings this month and YTD.
- Order list with per-order savings (no PII).

7.7.3 Additional product visibility

Travel agents have access to see a vast number of products which are not visible for standard customers.

→ Travel Agents can see B2B's vast portfolio.

7.8 UI Wireframe Outline (Text Wireframes)

Public pages

- /b2b/affiliate and /b2b/creators → program explanation + CTA apply
- /b2b/travel-agents → discount explanation + CTA apply

Apply form

- Fields: program selection (affiliate vs travel agent), subtype (affiliate only), name, email, website, social links, company details, optional payout details.

Partner portal — Affiliate

- NAV: Dashboard | Statements & Invoices | Payout Details | Account | Logout
- Dashboard:
 - Current month counters (reset after close)
 - Last closed month: invoice amount + statement download
 - Coupon/Deeplink + Promo kit (copy snippets; Deeplink Builder)
 - Transactions (no PII)

Partner portal — Travel agent

- NAV: Orders | Agent Discount & Savings | Account | Logout
- Savings page: rates + savings (month/YTD) + orders list (no PII)

Partner portal — Gooru Guide

- NAV: Topics | My Articles | Earnings | Media | Profile | Account | Logout
- Topics:
 - Available topics list (filtered to assigned city/cities)
 - Claim topic (shows TTL + rules)
- My Articles:
 - Draft / Submitted / Published (noindex) / Approved (index) / Needs fixes
 - Freshness status badge + next refresh due date
- Earnings:
 - Current month counters + last closed month (same statement mechanics as affiliate)
 - Per-article summary (no customer PII)
- Media:
 - Browse/search Rosotravel approved media assets
 - Insert/copy image references into articles
- Profile:
 - Bio, languages, city scope, social links

Admin / backoffice

Dedicated Partners section (separate from Bookings → Discount Codes):

- Global Settings
- Applications (in review)
- Affiliates
- Travel Agents

- Gooru Guides
- Topic Pool (seed/import + status management)
- Gooru Review Queue (SEO approvals to index)
- Media Library (approved assets + approvals workflow)
- Ledger
- Statements
- Invoices
- Payouts

7.9 API Endpoints (REST)

All endpoints are indicative; naming may follow existing conventions.

7.9.1 Public / tracking

- GET /b2b/affiliate
- GET /b2b/creators
- GET /b2b/travel-agents
- POST /api/partners/apply

Deeplink routing (preferred)

- GET /go/{deeplink_id}
 - sets attribution cookie
 - redirects to stored target_url (canonical allowlist enforced)

Optional legacy tracking route (if kept)

- GET /api/track/deeplink?pid=...&did=... (server-side cookie set; may also be client-side)

7.9.2 Auth

- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/forgot-password
- POST /api/auth/set-password (invite token flow)

7.9.3 Partner portal (Affiliate)

- GET /api/partner/me
- GET /api/partner/assets
- GET /api/partner/deeplinks
- POST /api/partner/deeplinks (create campaign deeplink with target entity allowlist validation)
- DELETE /api/partner/deeplinks/{deeplink_id} (revoke)
- GET /api/partner/balances (must return: current_month counters + last_closed_month summary)
- GET /api/partner/transactions?from=&to=&status=&source=
- GET /api/partner/statements
- GET /api/partner/statements/{id}/download?format=pdf|csv
- POST /api/partner/invoices (upload invoice)
- GET /api/partner/invoices
- PUT /api/partner/payout-details

7.9.4 Partner portal (Travel agent)

- GET /api/agent/discounts
- GET /api/agent/savings-summary
- GET /api/agent/orders?from=&to=

7.9.5 Gooru Guide portal

- GET /api/gooru/me
- GET /api/gooru/topics?city_id=&status=available
- POST /api/gooru/topics/{topic_id}/claim
- POST /api/gooru/topics/{topic_id}/release
- GET /api/gooru/articles
- POST /api/gooru/articles (create draft)
- PUT /api/gooru/articles/{id} (edit)
- POST /api/gooru/articles/{id}/submit
- GET /api/gooru/earnings-summary
- GET /api/gooru/media/search?q=&city_id=&type=

7.9.6 Checkout / orders (integration hooks)

- POST /api/checkout/validate-coupon (partner coupons only; returns discount + attribution)
- POST /api/checkout/create-session (captures attribution + applies travel-agent login discount if eligible)

Deterministic attribution (hard; stored per order item)

At checkout session creation (server-side), the system **MUST** resolve partner attribution deterministically in a single global order:

- 1) coupon_code (affiliate) wins and overrides all other attribution,
- 2) else deeplink_id cookie (affiliate) if present and valid,
- 3) else gooru_content cookie (guide_partner_id + article_id) if present and eligible,
- 4) else no attribution.

The resolved attribution MUST be stored as an immutable attribution_snapshot:

- at session level (for audit),
- and per order item (for economics), including:
 - campaign_type,
 - partner_id (or guide_partner_id),
 - coupon_code (if used),
 - deeplink_id (if used),
 - article_id (if gooru_content),
 - product_source (OWN|PARTNER),
 - rule_source_version (Partners global settings version).

Hard rule:

All discounts/commissions are computed per order item based on product_source, and mixed carts must apply item-by-item economics.

- POST /api/orders/{id}/mark-paid (internal hook; create ledger entries per item)
- POST /api/orders/{id}/refund (internal hook; reverse ledger entries)

Cancellation rule (hard):

No public “cancel by bot” endpoint. Expose only customer-driven cancellation flow:

- GET /api/orders/{id}/cancellation-link (returns Rosotravel-domain link for customer self-service)

7.9.7 Admin endpoints

- GET /api/admin/partners/settings
- PUT /api/admin/partners/settings (audited)
- GET /api/admin/partners/applications
- POST /api/admin/partners/applications/{id}/approve
- POST /api/admin/partners/applications/{id}/reject
- POST /api/admin/partners/create
- GET /api/admin/partners
- GET /api/admin/partners/{id}
- PUT /api/admin/partners/{id}

Admin — Gooru specifics

- POST /api/admin/gooru/create
- PUT /api/admin/gooru/{id} (assign cities, status)
- GET /api/admin/gooru/topics
- POST /api/admin/gooru/topics/import (optional)
- GET /api/admin/gooru/review-queue
- POST /api/admin/gooru/articles/{id}/approve-index
- POST /api/admin/gooru/articles/{id}/request-fixes

Ledger / statements / payouts

- GET /api/admin/ledger?partner_id=&period=
- POST /api/admin/statements/close?month=YYYY-MM (runs the monthly close; typically scheduled on day 7)
- GET /api/admin/statements
- GET /api/admin/invoices
- POST /api/admin/invoices/{id}/approve
- POST /api/admin/invoices/{id}/reject
- POST /api/admin/payouts/{statement_id}/mark-paid (store transfer reference)

7.10 Background Jobs and Emails

Jobs

- Earned conversion (daily): pending → earned when earned_at passed.
- Monthly close job (scheduled on day 7): locks previous month statement and resets portal counters for the new month display.
- Statement history maintenance (monthly close output stored immutably).
- Invoice deadline enforcement: invoices after deadline shift to next payout cycle.
- Refund/chargeback reconciliation handlers.

Gooru additions

- Topic Pool Generator (after each keyword retrieval/batch activation): derive informational blog topics from keyword_map.

- Topic claim TTL enforcement (daily): auto-release expired claims and unlock reservation (audited).
- Gooru freshness eligibility job (daily): mark articles eligible/ineligible based on last_guide_update_at (60-day rule).
- Freshness reminder notifications (every 21-days): notify guides approaching refresh due date.

Emails

- Application received (partner + internal)
- Approved + invite to set password
- Rejected
- Statement closed/generated (partner) (triggered by monthly close on day 7)
- Invoice approved/rejected
- Payout marked paid

Gooru additions

- Topic claimed confirmation (guide)
- Topic claim expiring soon (guide)
- Topic released due to TTL (guide)
- Article submitted (SEO team + guide)
- Article approved to index (guide)
- Article returned for fixes (guide)
- Freshness reminder (guide)

7.11 Security and Abuse Controls

- Apply endpoint rate limiting (+ CAPTCHA optional).
- Coupon brute-force protection (rate limit validation attempts).
- No customer PII in partner portal.
- Audit log for admin actions (settings, approvals, statement close, payout marked paid).
- Invite-based password setup (never email passwords).

Additional controls (Gooru + Deeplinks)

- Prevent open redirect: deeplink target_url must be stored server-side and validated against Rosotravel canonical allowlist.
- Rate limit deeplink creation and topic claiming to prevent abuse.
- Content submission anti-spam: per-guide daily submission caps + anomaly detection.
- Provenance audit: store editor identity, timestamps, and version history for all articles.
- Media licensing compliance: enforce Media Library-only in MVP; if uploads enabled later, require rights attestation + moderation queue.

7.12 Settlement Currency & FX Rule (hard: use site FX settings)

- Statements/payout thresholds are evaluated in EUR.
- If a sale is paid in non-EUR currency (e.g., USD), the ledger must store:
 - original currency + amount,
 - EUR-converted amount computed using the same FX configuration/rates used by the Rosotravel website/checkout for that currency pair at payment time.
- The system must persist:
 - fx_rate_used (numeric),

- fx_pair (e.g., EUR/USD),
- fx_timestamp (payment time),
- and a reference to the internal “site FX configuration version” if available.

VIII. Marketing loop and datalyer

XIX. Patch notes: Discovery Refinement Items

Purpose of this section

To avoid over-expanding the main specification before handoff, the items collected in this Patch Notes section describe topics that are directionally agreed, strategically important, and already conceptually defined, but still require additional refinement during the 3-week discovery phase.

These notes are intentionally kept outside the main chapter structure for now.

They should be treated as:

- approved design direction,
- implementation guidance,
- and discovery scope input,

but not yet as fully frozen low-level functional specification.

Why this section exists

Some areas of the project are already sufficiently mature at the conceptual level to be documented now, but would unnecessarily overload the core chapters if inserted immediately in full detail.

At the same time, these topics are too important to leave undocumented before project handoff.

This section therefore captures:

- agreed strategic direction,
- known implementation assumptions,
- open technical or UX questions that must be validated in discovery,
- and patch-level requirements that may later be integrated into the appropriate core chapters after discovery closes.

Interpretation rule

Unless explicitly stated otherwise:

- Patch Notes do not replace the main specification,
- they extend it,
- clarify it,
- or define areas that are intended for later integration into the main chapters.

Discovery expectation

During discovery, each patch item should be reviewed and finalized in terms of:

- technical feasibility,
- data availability,
- UX implications,
- governance and ownership,
- and final chapter placement.

After discovery, validated items from this section may be:

- moved into the correct chapters,
- expanded into full functional requirements,
- or partially revised where implementation constraints require it.

General principle

Patch Notes exist to keep momentum and preserve agreed thinking without forcing premature over-engineering of the core specification.

They ensure that important concepts are captured now, while still leaving space for controlled validation and refinement during discovery.

9.1 Reviews Loop, Verified Reviews, and Review Collection CRM Flow

Status: Conceptual patch for discovery validation

Purpose: To avoid expanding the core specification too early, the points below define a high-priority area that must be finalized during the 3-week discovery phase. These rules are directionally agreed and should be treated as implementation guidance and discovery scope, not yet as fully frozen low-level functional detail.

1) Goal of the Reviews Loop

Rosotravel must build a unified review system that:

- allows every customer who booked through Rosotravel to leave a review,

- works for both internal products and external products purchased on Rosotravel,
- clearly marks the source of every review,
- supports “verified by booking” trust logic,
- feeds city/product/attraction reputation signals,
- and provides data for the Human-Face / Latest Tour Snapshot layer.

This reviews loop is not only a content feature.

It is also:

- a trust engine,
- a CRM re-engagement flow,
- a source of recent proof,
- and a structured input for ratings, reputation, and snapshot components.

2) Core review eligibility rule

A review may be submitted only by a customer whose booking exists in Rosotravel systems.

This applies to:

- internal products,
- external / supplier products,

as long as the booking was completed through Rosotravel.

This means:

- Rosotravel must support verified review collection for both own and partner inventory sold on Rosotravel,
- but only when Rosotravel has a booking record.

Hard rule:

Every Rosotravel-submitted review must be marked as:

“Verified booking”

or equivalent visible label.

Verification basis:

- booking number / booking ID
- linked customer
- linked product
- linked order status

3) Review sources and source transparency

Rosotravel must support multiple review sources.

At minimum:

- Rosotravel reviews
- Viator reviews
- Tripadvisor reviews

Every review displayed on Rosotravel must visibly show its source.

Required source labels per review card:

- Source: Rosotravel
- Source: Viator
- Source: Tripadvisor

Important:

This source label must appear on every review, not only in schema or hidden metadata.

Why this matters:

- transparency,
- legal / trust clarity,
- cleaner LLM understanding,
- and proper separation between first-party and imported review evidence.

4) Verified status rules

Only Rosotravel-native reviews tied to a booking can carry the strongest verification label.

Recommended visible labels:

A) For Rosotravel review linked to booking:

- Verified booking
- Source: Rosotravel

B) For imported external reviews:

- Source: Viator
- Source: Tripadvisor
- optionally “Imported review” / “External review source”

Important:

Do not imply that imported external reviews are “verified by Rosotravel booking” unless the booking linkage exists on Rosotravel side.

5) Aggregate rating rules

Rosotravel may display a combined aggregate rating across multiple review sources.

This is allowed because:

- users care about total reputation,
- aggregate rating supports stronger destination and product trust signals,
- and it is commercially useful.

However, aggregate rating must follow clear rules:

- the product / city / country aggregate may combine Rosotravel + external review sources,
- but source-specific reviews must still remain individually labeled,
- and discovery must define whether weights are equal or if certain sources are normalized.

Current agreed direction:

- aggregate rating may be combined regardless of source,
- but transparency at review-card level is mandatory.

Discovery items to validate later:

- whether all sources have equal weight,
- whether source confidence multipliers are needed,
- whether source-specific counts should be visible near the aggregate,
- whether imported reviews require separate freshness handling.

6) Review submission flow (customer journey)

The review loop should work as follows:

Step 1 — booking completed

After a booking is completed and the experience date passes, Rosotravel becomes eligible to request a review.

Step 2 — review request email

Customer receives an email asking for a review.

Email must include:

- product name
- destination / tour context
- clear CTA to leave a review
- 1–5 star buttons directly inside the email

Step 3 — star click from email

When the customer clicks one of the stars (1–5), they are redirected to a dedicated review landing page.

Important:

The clicked star value should be passed through the URL/session and prefilled on the landing page.

Step 4 — dedicated review landing page

The landing page must:

- be tied to the exact booking,
- prefill the product context,
- prefill the star rating if selected from email,
- allow the customer to complete the review,
- optionally allow photo / video upload,
- clearly state that the review will be marked as verified if booking linkage is confirmed.

Step 5 — review saved

Once submitted:

- the review is linked to the booking,
- linked to the product,
- linked to the city / attraction graph where relevant,
- and becomes eligible for moderation / publishing / snapshot clustering.

7) Review landing page concept

The review landing page should be a dedicated conversion page for feedback collection.

It should include:

- product title
- city / destination
- selected star rating
- review text area
- optional media upload
- confirmation that this review is linked to a verified booking
- short privacy / moderation note

Important:

The exact review form fields are intentionally left for discovery refinement later.

Known direction:

- review fields exist,
- media upload should be supported,
- booking linkage is mandatory,
- detailed form UX can be finalized in discovery.

8) CRM reminder loop

The review request flow must be integrated with CRM / lifecycle automation.

Agreed direction:

Rosotravel should be able to send up to 4 review reminder messages until the customer submits a review.

Rules:

- message sequence stops as soon as a review is submitted,
- each message is tied to the booking / product,
- each message can be delayed by configured intervals,
- cadence and exact copy are to be defined in discovery.

Recommended concept:

- Message 1: first review request
- Message 2: gentle reminder
- Message 3: stronger trust/value reminder
- Message 4: final reminder

Discovery items:

- exact send timing after tour date,
- suppression rules,
- localization,
- email vs SMS / WhatsApp support,
- whether 1-star / 2-star customers should be routed differently.

9) Review loop and Human-Face / Snapshot connection

This reviews loop is strategically important because Rosotravel-native reviews linked to bookings can power:

- Latest Tour Snapshot
- Recent guest moments
- review freshness windows
- destination-level recent proof
- attraction-level recent traveler sentiment

This is one of the strongest reasons to build Rosotravel-native reviews even for external products sold on Rosotravel.

Why:

- it provides trusted first-party review timestamps,
- gives a better source for review-date clustering,
- improves recent proof quality,
- and reduces dependence on imported external review structures.

10) Media upload in reviews

Rosotravel review submission should allow customers to upload:

- photos
- and videos

These media assets should become eligible for:

- product-level review gallery
- Latest Tour Snapshot / Recent Experience blocks
- city-level recent media strips
- attraction-level recent traveler moments

Hard rule:

Customer review media must be moderated before public display.

Recommended metadata per uploaded media:

- media_id
- linked_review_id
- linked_booking_id
- linked_product_id
- source_type = customer_review
- uploaded_at
- moderation_status

11) Imported reviews vs Rosotravel reviews

Rosotravel must distinguish clearly between:

A) Rosotravel-native reviews

- linked to booking
- can be verified
- can power stronger snapshot logic

- can accept customer-uploaded media

B) Imported external reviews

- Viator / Tripadvisor / other approved sources
- source-labeled
- included in reputation and aggregate rating
- may support text/theme/rating usage depending on source terms and ingestion model

Important:

Imported review freshness and Rosotravel-native review freshness should not be treated as identical by default.

This must be validated in discovery.

12) Review moderation and governance

All reviews must pass through moderation and governance rules.

Required concepts:

- pending / approved / rejected / hidden
- spam detection
- abuse handling
- duplicate detection
- source integrity
- media moderation

Discovery must clarify:

- whether moderation is pre-publication or partially post-publication,
- whether negative reviews have escalation rules,
- whether supplier-dispute flow is needed for external-product reviews.

13) Review object — conceptual data model

Recommended direction for discovery:

review

- review_id
- booking_id (nullable only for imported reviews)
- product_id
- city_id
- attraction_ids[]
- source_type (Rosotravel / Viator / Tripadvisor / other)
- verification_type (booking_verified / imported / unknown)
- rating
- title (optional; discovery)
- review_text
- submitted_at
- experience_date (preferred when known)
- published_at
- moderation_status
- media_ids[]

- reviewer_display_name
- reviewer_country (if available and compliant)

14) Visible review card requirements

Every public review card must clearly display:

- rating
- review text
- traveler name/display name
- date
- source label
- verified status (if Rosotravel booking verified)

Recommended visible example:

★★★★★

“Very well organized and easy to follow.”

Anna, Germany

March 2026

Verified booking • Source: Rosotravel

or

★★★★★

“Great guide and smooth entry.”

March 2026

Source: Viator

15) Relationship with aggregate ratings on product / city / country pages

This review loop feeds:

- product aggregate rating
- city aggregate rating
- country aggregate rating
- “What travelers value most” destination themes
- snapshot freshness components

Important:

The review system must be built so that aggregation logic can operate at:

- product scope
- city scope
- country scope
- attraction scope (if needed)

16) Discovery topics explicitly left open

The following must be finalized during discovery:

- exact review form fields
- rating weighting logic across sources
- moderation workflow details
- imported review ingestion governance
- reminder timing and CRM cadence

- review publication SLA
- media/video limits and moderation
- routing for low ratings / service recovery workflow
- legal/compliance notes for imported sources and source labeling

17) Why this patch matters

This loop is strategically critical because it connects:

- CRM
- trust
- ratings
- recent proof
- Human-Face layer
- destination reputation
- and long-term SEO / LLM brand grounding

Without this loop, Rosotravel will rely too heavily on imported reviews.

With this loop, Rosotravel builds its own verified review graph across both internal and external products sold on Rosotravel.

[PATCH NOTES] — Performance, Async Processing, Backup, and Voucher Adjustments

Status: Immediate implementation patch

Purpose: Add final operational requirements that should be implemented now, without expanding the main specification excessively.

9.2 Web performance / Core Web Vitals

The platform must be built and optimized to meet current Google page experience standards on both mobile and desktop.

Minimum target standard:

- LCP $\leq$ 2.5s
- INP $\leq$ 200ms
- CLS $\leq$ 0.1

Measurement rule:

- success should be evaluated primarily on real-user data at the 75th percentile,
- and additionally monitored in PageSpeed Insights / Lighthouse for both mobile and desktop environments.

Internal launch target:

- Lighthouse Performance score 90+ on key public templates wherever realistically achievable,
- with special priority on City, Things-to-do, Attraction, Product, Bundle, and Explore pages.

Important:

Performance must be treated as part of rendering architecture, media policy, and component design, not as a late optimization step.

9.3 Backend responsiveness and async processing

The backend must remain responsive during heavy operations.

Longer-running tasks must not block the user interface.

Hard rule:

If an operation takes longer than 5 seconds, it should be moved to background processing / queue logic.

Examples:

- AI generation
- translation
- media processing
- video rendering
- product ingestion / enrichment
- bulk updates
- large imports

User-facing requirement:

- the user must not be blocked from continuing other work while processing runs,
- the UI must show that the process is running,
- and the UI should show the current processing stage / status where possible.

Required statuses (minimum concept):

- queued

- processing
- completed
- failed

Recommended rule:

- short operations may run inline only if they reliably complete in under 5 seconds,
- everything slower should run asynchronously.

Queue safety requirement:

The system may impose limits on the number of concurrent heavy jobs per user / per workspace / per queue type in order to protect platform responsiveness and avoid UI degradation.

9.4 Backup and restore

The platform must include independent backup protection.

Minimum backup rule:

- full backup at least once per week
- retain backups for the last 4 months

Storage rule:

Backups must be stored outside the primary application environment on an independent storage target.

Preferred model:

- one primary backup location
- one secondary backup location

Accepted examples:

- Google Workspace / Google Drive environment controlled by Rosotravel
- separate cloud storage bucket
- separate independent backup server

Important:

Backups should not depend only on the same production environment they protect.

Restore requirement:

The development / operations team must have the ability to restore backups when needed.

Restore capability must be tested, not assumed.

9.5 Voucher and output-format adjustments

There will be additional changes to vouchers and selected output rules compared with the current version.

This includes, at minimum:

- visual / stylistic voucher changes,
- updated labels / wording,
- adjusted product code handling and retrieval,

- and other small output-format improvements aligned with the new product and booking logic.

This item is intentionally kept high-level here and should be finalized during implementation together with the relevant booking / product output screens.

X. Datalayer and CRM loop

TRACKING, CRM INTEGRATION, AUDIENCE DATA LAYER, AND
RETENTION READINESS

(Rosotravel — First-Party Customer Data and Growth Architecture)

1. Purpose

The new Rosotravel website must not treat tracking as a simple analytics or ad-pixel implementation.

It must be designed as a first-party customer data and retention architecture.

The purpose of this layer is to create one consistent system that connects:

- page behavior,
- destination interest,
- product interaction,
- booking behavior,
- lead capture,
- CRM enrichment,
- retargeting,
- and lifecycle communication.

This layer must support:

- analytics,
- paid media measurement,
- server-side conversion tracking,
- CRM and retention automation,
- audience building,
- identity stitching,
- and decision-platform learning.

Without this layer, Rosotravel will only measure traffic and transactions.

With this layer, Rosotravel can understand who the traveler is, what destination or experience they care about, what they nearly bought, what they requested, and how to bring them back.

CRM Integration Principle

The Rosotravel platform must be designed from the start to support easy integration with an external CRM / marketing automation platform, such as HubSpot or an equivalent solution.

This means the website, backend, and event architecture must be CRM-ready by design.

Rosotravel's core system should own:

- event collection,
- booking and lead source-of-truth,

- consent state,
- identity stitching,
- destination and product interest data,
- and trigger generation.

The external CRM / marketing automation platform should own:

- email composition,
- lifecycle workflows,
- segmentation,
- nurture logic,
- reminder sequences,
- campaign delivery,
- and contact-level marketing automation.

The implementation must therefore support straightforward integration with HubSpot or another comparable CRM through:

- API endpoints,
- webhooks,
- event sync,
- contact property sync,
- lifecycle trigger sync,
- and near-real-time or scheduled data synchronization.

Hard requirement:

The new architecture must not depend on a custom-built internal email automation system in order to run retention, nurture, and review-request workflows.

Instead, it must expose the right data and triggers so that an external CRM can operate reliably and with minimal custom integration effort.

Examples of supported CRM-trigger use cases:

- lead created
- purchase completed
- private request created
- itinerary request submitted
- review request eligible
- abandoned planning flow
- destination/product interest update

This should be treated as a core architecture requirement, not as a later optional integration.

2. Strategic principle

The current website data layer is largely commerce-event oriented.

That model is not sufficient for the new Rosotravel platform.

The new platform requires:

- A) destination-aware event tracking,
- B) page identity tracking,

- C) audience and intent tracking,
- D) server-side conversion truth,
- E) CRM sync and identity stitching,
- F) retention-ready lead and purchase enrichment.

This means the new data layer should be treated as a new architecture layer, not as a light patch on top of the old one.

3. What to do with the current data layer

The current website implementation should be treated as a legacy reference, not as the final architecture for the new frontend.

Recommended migration rule:

- keep the current data layer only as a temporary input for audit and migration mapping,
- do not copy it 1:1 into the new frontend,
- rebuild the event contract from scratch around the new page types, entity model, and retention requirements.

Why:

The current implementation is useful as a source of:

- event naming history,
- pixel setup visibility,
- current gaps,
- and migration comparison.

But it is not sufficient as the core contract for the new site because it is too narrowly focused on product-commerce events and does not capture enough destination, audience, and lifecycle context.

Therefore:

the old data layer should be:

- audited,
- mapped,
- and selectively migrated,

but the new site should run on a new event contract.

4. Event contract — single source of truth

The new frontend must implement one unified event contract that is independent of UI layout details.

This event contract must be the source of truth for:

- GA4
- Google Ads
- Meta Pixel
- Meta Conversions API
- CRM triggers
- internal analytics
- retargeting audiences
- and funnel dashboards

The event contract must not be defined ad hoc by individual frontend components.

5. Minimum event model

At minimum, the new website must support the following events:

Page and destination events

- page_view
- view_country
- view_region
- view_city
- view_attraction
- view_product
- view_bundle
- view_near_page
- view_blog
- view_expert
- view_explore

Selection and intent events

- select_product
- select_bundle
- select_rail
- select_intent

- select_audience_segment
- select_filter
- select_private_upgrade
- select_more_options

Commerce events

- add_to_cart
- remove_from_cart
- begin_checkout
- add_payment_info (if checkout structure includes that step)
- purchase

Lead and request events

- lead_capture
- submit_private_request
- submit_itinerary_request
- submit_contact_request
- signup
- login

Optional but highly recommended

- save_plan
- save_product
- compare_products

- quiz_start
- quiz_complete
- review_request_sent
- review_submitted

6. Required parameters for all events

Every event must include, where applicable:

Identity and page context

- page_type
- entity_id
- page_url
- canonical_target or canonical page identity
- language
- currency
- device_type

Acquisition context

- traffic_source
- utm_source
- utm_medium
- utm_campaign
- utm_content
- utm_term

- referrer_class

User and session context

- user_id (if authenticated)
- anonymous_id (if anonymous)
- session_id
- consent_state

Decision-platform context

- audience_segment
- audience_segment_selected
- intent
- constraints (if relevant)
- decision_mode / planner_mode / compare_mode (if implemented)

Product and portfolio context

- product_archetype
- value_tier
- city_interest
- destination_id
- destination_type

This is required because Rosotravel must understand not only that a user viewed a product, but also:

- which destination they cared about,
- what kind of traveler they behaved like,
- and how that should later influence CRM, retargeting, and decision logic.

7. Page identity and SSOT connection

Every tracked page must be tied to the system identity model.

Required page identity dimensions:

- entity_id
- page_type
- owner_url
- canonical_target
- index_state
- hreflang_cluster_id

This is needed for:

- migration safety,
- SEO-safe analytics,
- deduplication,
- CRM segmentation,
- and downstream AI / LLM surfaces.

Without stable page identity, traffic and conversion analysis across a programmatic site becomes unreliable.

8. Product and list event rules

The legacy questions around `view_item_list`, `view_item`, `item_id`, `currency`, `price`, and `quantity` should not be handled ad hoc.

They should be explicitly standardized in the new event contract.

8.1 Homepage listing logic

If product cards are shown on the homepage inside curated modules, Rosotravel should decide explicitly whether those modules count as product-list impressions.

Recommended rule:

- homepage product rails may trigger list-view tracking only if they are treated as genuine commerce list surfaces,
- otherwise they should use a separate curated-list event model (preferred for decision-platform clarity).

Recommended direction:

Use a dedicated curated-list event model for homepage and destination decision rails instead of forcing all surfaces into generic ecommerce `view_item_list` semantics.

8.2 Product listing pages

Shop pages, destination listing pages, and other controlled listing pages should use one consistent structure.

Dynamic loading / “See more” interactions should trigger incremental list exposure events, not pretend that all items were visible on initial load.

Recommended rule:

- fire initial list-view event for first rendered items only,
- fire additional list-view / list-extend events for newly revealed items.

8.3 Localization

Localized versions may use translated text fields, but identity fields must remain stable across languages:

- same entity_id
- same product identity
- same canonical logic
- translated display fields only where appropriate

8.4 Currency

Displayed and tracked prices should reflect the user-facing selected currency for frontend events.

A base / accounting currency may still be stored server-side for reporting reconciliation.

8.5 Item identity

If no formal SKU exists, product reference code may be used as item_id.

Fallback logic must be standardized and stable.

Recommended rule:

- primary item_id = stable product reference code
- fallback item_id = controlled internal format based on stable product ID

The key requirement is stability across all systems.

8.6 Quantity

For view events, quantity should remain 1 unless the UX already contains a concrete quantity/participant selection.

For action events:

- add_to_cart
- begin_checkout
- purchase

quantity must reflect the actual selected purchase quantity logic.

8.7 Product duplication across sections

If one product appears in multiple decision sections, the system should not flatten all sections into one ambiguous list.

Instead, it should preserve section context.

Recommended direction:

- track section / rail context explicitly,
- preserve list_name / module_name / slot context,
- and allow the same product to be seen in multiple contexts if that reflects real UX.

9. Server-side tracking and conversion truth

Rosotravel must implement server-side measurement for critical conversions.

Required direction:

- GTM Server-Side or an equivalent server-side event gateway
- Meta Conversions API

- Google Ads Enhanced Conversions
- backend-confirmed purchase event as the conversion source of truth

Hard rule:

Purchase must not depend only on browser-side tracking.

Recommended model:

- browser events for UX behavior and partial funnel visibility
- server-side events for critical conversion confirmation
- deduplication between browser and server where required

10. Google stack implications

The new implementation will likely require updates and restructuring in:

- GA4 property event definitions
- Google Tag Manager setup
- Google Ads conversion mapping
- enhanced conversion setup
- tag governance
- and debug / QA process

Recommended implementation direction:

A) Rebuild the GA4 event architecture around the new event contract

B) Rebuild or substantially rework the GTM implementation to support:

- page identity parameters

- destination context
- event normalization
- server-side routing

C) Update Google Ads event mapping to align with the new conversion model

D) Validate whether first-party delivery or gateway patterns are beneficial

E) Maintain one conversion truth for purchase and key lead events

11. Meta stack implications

The new implementation will likely require updates and restructuring in:

- Meta Pixel event mapping
- Meta CAPI server-side event delivery
- deduplication logic
- event parameter consistency
- and audience-building logic

Recommended model:

- browser Pixel + server-side CAPI working together
- same core events sent where relevant
- event_id-based deduplication
- purchase, lead, private inquiry, and other critical events aligned across systems

12. Analytics engineering support requirement

The current data layer code was reportedly prepared by a marketing specialist from Appinventiv.

That may be sufficient for a basic pixel or ecommerce setup, but the new architecture is broader and more technical.

Therefore, Rosotravel should explicitly ask the vendor whether they have in-house capability for:

- analytics engineering,
- data layer architecture,
- GTM / GA4 redesign,
- server-side tagging,
- conversion debugging,
- and CRM event integration.

If not, the vendor should confirm whether they require support from:

- a dedicated analytics implementation specialist,
- a GTM / server-side tracking specialist,
- or Rosotravel's own marketing / analytics advisor.

This should be clarified early.

Otherwise there is a risk that the frontend is built correctly while the data layer remains weak or inconsistent.

13. Consent mode and privacy

The new website must support:

- consent banner
- consent mode

- analytics vs marketing separation
- event behavior under no-consent scenarios
- lawful storage and activation logic for identifiers

Consent status must be available to the event pipeline and CRM sync layer.

This is especially important in the EU context.

14. CRM integration layer

CRM must be treated as an external connected system, not as something rebuilt entirely from scratch inside Laravel admin.

The website / backend must provide a CRM integration layer that supports:

- lead_created webhook / event
- purchase_completed webhook / event
- private_request_created webhook / event
- itinerary_request_created webhook / event
- nightly sync or near-real-time sync for contact properties and status updates

Minimum CRM contact fields

- email
- phone (if collected)
- consent status
- language
- acquisition source

- UTM data
- lifecycle stage
- city_interest
- last_seen_city
- last_seen_products
- last_seen_attraction
- strongest intent
- strongest audience signal
- purchase status / purchase history summary

15. Identity stitching

Identity stitching is critical.

The system must be able to:

- track browsing on anonymous_id
- later connect it to email or user_id after lead capture / purchase / login
- preserve the relationship between pre-conversion browsing and known contact profile

This allows:

- proper retargeting
- destination-specific CRM journeys
- abandoned planning flows
- product recommendation emails

- and post-lead nurture based on actual browsing interest

Recommended rule:

anonymous browsing history must not be lost once the user becomes known.

16. Hooks, lead capture, and CRM orchestration

Hooks are not just frontend modules.

They are trigger points into the CRM and retention machine.

Therefore, all major lead hooks must be:

- CMS-controlled
- event-tracked
- and CRM-connected

Examples:

- newsletter signup
- itinerary / plan capture
- private inquiry
- destination PDF
- bundle save / email me this plan

Flow concept

A) user interacts with hook

B) event is sent through event contract

- C) lead is created or updated
- D) CRM receives contact + context
- E) CRM runs the appropriate automation or email sequence
- F) future emails use destination/product context from the data layer

17. How CRM should use this data

CRM should not send generic emails only.

It should use tracked context to send:

- relevant destinations
- relevant cities
- recently viewed products
- relevant bundles
- private-tour alternatives
- stronger trust / proof content
- reminder or nurture sequences matched to the user journey

Examples:

If a user viewed:

- London city page
- Westminster Abbey attraction page
- two London bundles
- one private request module

CRM should be able to send:

- London-focused follow-up
- best-fit London products
- private or premium alternatives
- bundle reminder
- “continue planning” message

This is why city_interest, product interest, and intent must be in the data layer from the start.

18. On-site conversion modules and hook governance

Every major page type must support CMS-controlled hooks and trust modules.

Required module classes:

- lead magnet module
- trust module
- FAQ module
- cross-sell module
- private upgrade module
- CTA rail module
- sticky mobile CTA
- planning/save module

Requirement:

marketing must be able to:

- enable/disable them
- edit copy
- choose variants
- and use them in experiments

without engineering work for every small change.

19. Audience and retargeting readiness

The data layer must support retargeting that is not flat.

Required dimensions for retargeting readiness:

- audience_segment
- intent
- product_archetype
- value_tier
- city_interest
- destination interest
- planner / compare signals

Without this, retargeting becomes generic and expensive.

With this, Rosotravel can build:

- city-specific audiences
- attraction-specific audiences
- private-tour interest audiences

- first-timer vs deep-dive audiences
- planner vs quick-buyer segments

20. SEO migration safety and tracking continuity

The migration must preserve both SEO equity and analytics continuity.

Required migration layer:

- redirect_map table
- old_url → new_url mapping
- pattern-based rules + exceptions
- 404 monitoring
- preserved query params where appropriate
- no accidental loss of attribution data

The system must preserve:

- UTM handling
- page identity continuity
- conversion tracking continuity
- canonical enforcement
- noindex enforcement
- hreflang continuity

Tracking and migration cannot be separated.

If migration breaks page identity, analytics and retargeting will also degrade.

21. Required CMS and operational views

At minimum, the operational system must support views or panels for:

- Entity Registry
- Batch Processing
- Page Editor
- Redirect Manager
- SEO Governance Monitor
- Review / request flow visibility
- optionally Review Engine / review request panel

These views are important not only for SEO and content operations, but also for marketing confidence and retention workflows.

22. Dashboard and reporting expectation

The new website is not marketing-ready unless a basic funnel and audience reporting layer exists.

At minimum, Rosotravel should be able to report:

- page views by page type
- city / attraction / product interest
- product selection from rails and shortlists
- checkout starts
- purchases
- leads by hook type

- private requests
- segment / intent distribution
- source / campaign performance
- funnel drop-off by destination and page type

23. Definition of done

The new website should be considered marketing-ready only if all of the following are true:

Tracking and measurement

- the event contract is implemented
- GA4 receives the correct events and parameters
- Google Ads conversion mapping is updated
- Meta Pixel and Meta CAPI are implemented consistently
- critical events are server-side supported where required

Identity and CRM

- lead capture writes to CRM
- purchase can sync to CRM
- private inquiry / itinerary request can sync to CRM
- anonymous browsing can be stitched to known contact identity
- consent state is respected across sync and automation

On-site marketing readiness

- hooks are CMS-controlled
- trust and CTA modules are configurable
- city pages support audience-aware rendering without new URLs
- destination and product interest can feed lifecycle logic

SEO-safe operations

- redirect engine works
- 404 monitoring exists
- canonical/noindex logic is deterministic
- page identity is stable and tracked

Reporting

- a baseline dashboard exists for funnel and retention readiness

24. Vendor implementation questions that must be clarified

The vendor must confirm:

A) Whether they will rebuild the analytics layer from scratch or adapt the legacy one

B) Whether they have in-house capability for:

- GTM / GA4 implementation
- server-side tagging
- Meta CAPI
- Google Ads enhanced conversions

- CRM event integration

C) Whether additional analytics implementation support is required

D) How they will connect frontend events, backend confirmations, and CRM sync

E) How event QA and debugging will be handled before launch

25. Final recommendation

Rosotravel should not treat this as “just data layer work”.

This is the first-party growth infrastructure of the new platform.

The correct framing is:

Tracking, CRM Integration, Audience Data Layer, and Retention Readiness

because the purpose is not only to measure what happened,

but to build the system that allows Rosotravel to:

- understand traveler intent,
- connect behavior with destination interest,
- follow up intelligently,
- retarget efficiently,
- and create a durable retention machine.

